

ក្រសួងអប់រំ យុវជន និងកីឡា

វិទ្យាស្ថានបច្ចេកវិទ្យាកម្ពុជា



ដេប៉ាតឺម៉ង់ ទេពកោសល្យគណិតវិទ្យាអនុវត្ត និងស្ថិតិ

របាយការណ៍ចុះកម្មសិក្សា វិស្វកម្មឆ្នាំទី៤

ប្រធានបទ: **ការបង្កើត** Dashboard **គ្រប់គ្រងចំណូល និងចំណាយដោយ ស្វ័យប្រវត្តិដោយប្រើប្រាស់** Python **និង** Excel

និស្សិត : **នៅ បញ្ញាវឌ្ឍន**

ឯកទេស : **វិទ្យាសាស្ត្រទិន្នន័យ**

គ្រូទទួលបន្ទុក : **លោក ឌន ដាវីន**

ឆ្នាំសិក្សា : **២០២៥-២០២៦**

MINISTERE DE L'EDUCATION,

DE LA JEUNESSE ET DES SPORTS

INSTITUT DE TECHNOLOGIE DU CAMBODGE

DEPARTMENT DE MATHEMATIQUES APPLIQUEES

ET STATISTIQUES

RAPPORT DE STAGE D'INGÉNIEUR

DE QUATRIÈME ANNÉE

Titre: Développement d'un Tableau de Bord Automatisé des Revenus et Dépenses à l'aide de Python et Excel

Etudiant : **NOV Panhavath**

Spécialité : **Science des Données**

Tuteur de stage : **Mr. Dorn Dawin**

Année scolaire : **2025-2026**



ក្រសួងអប់រំ យុវជន និងកីឡា

វិទ្យាស្ថានបច្ចេកវិទ្យាកម្ពុជា



ដេប៉ាតឺម៉ង់ ទេពកោសល្យគណិតវិទ្យាអនុវត្ត និងស្ថិតិ

របាយការណ៍ចុះកម្មសិក្សា វិស្វកម្មឆ្នាំទី៤

របស់និស្សិត: ដៅ បញ្ញាវឌ្ឍន

កាលបរិច្ឆេទការពារនិក្ខេបបទ: ថ្ងៃទី ខែ ឆ្នាំ ២០២៦

អនុញ្ញាតឲ្យការពារគម្រោង

ឆាយកវិទ្យាស្ថាន: ឯកឧត្តមសាស្ត្រាចារ្យបណ្ឌិត ប៉ូ គឹមថូ

ថ្ងៃទី ខែ ឆ្នាំ ២០២៦

ប្រធានបទ: ការបង្កើត Dashboard គ្រប់គ្រងចំណូល និងចំណាយដោយ

ស្វ័យប្រវត្តិដោយប្រើប្រាស់ Python និង Excel

សហគ្រាស: គ្រឹះស្ថានមីក្រូហិរញ្ញវត្ថុ មាតុភូមិ

ប្រធានដេប៉ាតឺម៉ង់: បណ្ឌិត លិន មន្តលសិរី_____

សាស្ត្រាចារ្យជឹកនាំគម្រោង: លោក ឌន ជាវីន_____

អ្នកទទួលខុសត្រូវក្នុងសហគ្រាស: លោកស្រី ភឌ្យ សុម៉ាលី_____

រាជធានីភ្នំពេញ



MINISTERE DE L'EDUCATION,
DE LA JEUNESSE ET DES SPORTS



INSTITUT DE TECHNOLOGIE DU CAMBODGE

DEPARTMENT DE MATHEMATIQUES APPLIQUEES ET STATISTIQUES

RAPPORT DE STAGE D'INGÉNIEUR DE QUATRIÈME ANNÉE

DE M. NOV Panhavath

Date de soutenance : le 2026

« Autorise la soutenance du mémoire »

Directeur de l'Institut: H.E. Prof. Dr. Po Kimtho

Phnom Penh, le 2026

Titre: Développement d'un Tableau de Bord Automatisé des Revenus et Dépenses à l'aide de Python et Excel

Etablissement du stage: **MEATOPHUM Microfinance**

Chef du département: **Dr. LIN Mongkolsery**_____

Tuteur de stage: **Mr. DORN Dawin**_____

Responsable de l'établissement: **Ms. WONG Somaly**_____

PHNOM PENH

INTERNSHIP REPORT ON

"Development of an Automated Income and Expense Dashboard Using Python and Excel"

at

MEATOPHUM Microfinance

Submitted By:

Nov Panhavath

Under the supervision of:

Ms. Wong Somaly

Under the advisory of:

Mr. Dorn Dawin

Date of defense: Day, 2026

In partial fulfillment of the requirements for the Internship Program

for the award of the degree of

Engineering in Applied Mathematics and Statistics

Majoring in Data Science

Department of Applied Mathematics and Statistics

Institute of Technology of Cambodia

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me throughout this internship and the completion of this report.

First, I would like to thank **H.E. Dr. LIN Mongkolsery**, Head of the Department of Applied Mathematics and Statistics at the Institut de Technologie du Cambodge, for his guidance and support throughout my academic journey.

I would also like to extend my heartfelt appreciation to **Mr. Dorn Dawin**, my thesis advisor, for the valuable guidance, feedback, and encouragement provided throughout the planning and writing of this report.

My sincere thanks go to **Ms. Wong Somaly**, Head of Finance at MEATOPHUM Microfinance, for giving me the opportunity to complete my internship at the organization, and for the guidance, trust, and practical knowledge shared with me throughout the project.

I am also grateful to the staff of MEATOPHUM Microfinance for their cooperation and for creating a welcoming environment that made this internship a valuable learning experience.

Finally, I would like to thank my family and friends for their continuous encouragement, patience, and support throughout my studies and this internship period.

អត្ថបទសង្ខេប

របាយការណ៍នេះបង្ហាញពីការងារដែលបានបញ្ចប់ក្នុងអំឡុង ពេលហាត់ការជាអ្នកវិភាគទិន្នន័យ (Data Analyst) នៅគ្រឹះស្ថានមីក្រូហិរញ្ញវត្ថុ មាតុភូមិ (MEATOPHUM Microfinance) ដែលជាស្ថាប័នហិរញ្ញវត្ថុមួយមានមូលដ្ឋាននៅរាជធានីភ្នំពេញ ប្រទេសកម្ពុជា ក្រោមការត្រួតពិនិត្យពីប្រធានផ្នែកហិរញ្ញវត្ថុ។ ការហាត់ការនេះផ្តោតលើការបង្កើតផ្ទាំងគ្រប់គ្រង (Dashboard) ចំណូល និងចំណាយដោយស្វ័យប្រវត្តិ ដើម្បីគាំទ្រដល់ការរាយការណ៍ហិរញ្ញវត្ថុប្រចាំខែនៅទូទាំង ១៩ សាខារបស់ស្ថាប័ន។

មុនពេលគម្រោងនេះ របាយការណ៍ចំណូលនិងចំណាយប្រចាំខែត្រូវបានចងក្រងឡើងដោយដៃ៖ តួលេខពីការទាញយកទិន្នន័យឆៅ (raw export) របស់សាខានីមួយៗ ត្រូវបានវាយបញ្ចូលឡើងវិញទៅក្នុងសន្លឹកសង្ខេបទិន្នន័យ(summary sheet) ជារៀងរាល់ខែ ដែលជាដំណើរការមួយស៊ីពេលច្រើន និងងាយនឹងមានកំហុសដោយសារមនុស្ស។ គោលបំណងនៃគម្រោងនេះគឺដើម្បីជំនួសលំហូរការងារដោយដៃនេះជាមួយនឹងផ្ទាំងគ្រប់គ្រង Excel ដែលមានរចនាសម្ព័ន្ធត្រឹមត្រូវ និងធ្វើបច្ចុប្បន្នភាពដោយខ្លួនឯង ដែលអាចធ្វើឲ្យស្រស់ (refresh) ដោយគ្រាន់តែចុចតែម្តងនៅពេលមានទិន្នន័យប្រចាំខែថ្មី។

ដើម្បីសម្រេចបានគោលដៅនេះ ប្រព័ន្ធរៀបចំទិន្នន័យ(data cleaning pipeline) ដោយផ្អែកលើ Python ត្រូវបានបង្កើតឡើង ដើម្បីទាញយក និងរៀបចំទម្រង់ទិន្នន័យឆៅប្រចាំខែទៅជាតារាងដែលមានរចនាសម្ព័ន្ធ និងស្អាត។ ក្នុងអំឡុងពេលនៃការអភិវឌ្ឍន៍ បញ្ហាគុណភាពទិន្នន័យមួយត្រូវបានរកឃើញ និងដោះស្រាយ៖ តារាងដែលគ្មានកាលបរិច្ឆេទមួយដែលជាប់នៅក្នុងឯកសារដើម បានបណ្តាលឱ្យតួលេខចំណាយមួយចំនួនត្រូវបានកត់ត្រាស្ទួន; ការត្រួតពិនិត្យសុពលភាព (validation check) ត្រូវបានបន្ថែមទៅក្នុងប្រព័ន្ធដើម្បីស្វែងរក និងដកចេញនូវភាពមិនប្រក្រតីបែបនេះដោយស្វ័យប្រវត្តិ។ បន្ទាប់មក ទិន្នន័យដែលបានសម្អាតរួចត្រូវបានបញ្ចូលទៅក្នុង Excel ដោយប្រើ Power Query និងរៀបចំជាទម្រង់គំរូទិន្នន័យផ្កាយ (star schema data model) ដែលមានតារាង Fact មួយ និងតារាង Dimension បី (សាខា កាលបរិច្ឆេទ និងគណនី)។ ជួរឈរតាមលក្ខខណ្ឌ (Conditional Column) ដោយផ្អែកលើការផ្គូផ្គងពាក្យគន្លឹះ (keyword matching) នៅក្នុង Power Query ត្រូវបានប្រើ ដើម្បីដាក់ក្រុមចំណាយឆៅជាងហុកសិបមុខ ទៅជាដប់ពីរប្រភេទដែលអាចយល់បានយ៉ាងងាយស្រួលសម្រាប់គោលបំណងរាយការណ៍។ បន្ទាប់មក រូបមន្ត Data Analysis Expressions (DAX) ត្រូវបានបង្កើតឡើងដើម្បីគណនាសូចនាករហិរញ្ញវត្ថុសំខាន់ៗ រួមមាន ចំណូលសរុប ចំណាយសរុប ចំណូលសុទ្ធ និងប្រាក់ចំណេញ (Profit Margin)។ សៀវភៅការងារ (workbook) ត្រូវបានរៀបចំជាបីសន្លឹកចម្បង៖ សន្លឹក Analysis សម្រាប់បង្កើត និងសាកល្បង Pivot Table សន្លឹក Dashboard ដែលបង្ហាញ Pivot Charts និងកាតសូចនាករប្រតិបត្តិការសំខាន់ៗ (KPI) ដែលអាចធ្វើអន្តរកម្មបាន និងសន្លឹក Summary ដែលបង្ហាញឡើងវិញនូវទម្រង់របាយការណ៍ចំណូលតាមសាខា។ ដើម្បីបង្កើនភាពងាយស្រួលប្រើប្រាស់ ប៊ូតុងរូបភាពដែលអាច

ចុចបាន (clickable image-based buttons) ត្រូវបានបន្ថែមទៅក្នុង Dashboard ដោយប្រើ VBA macro និងតំណភ្ជាប់ (hyperlinks) ដែលអនុញ្ញាតឱ្យអ្នកប្រើធ្វើបច្ចុប្បន្នភាពទិន្នន័យទាំងអស់ដោយគ្រាន់តែចុចម្តង និងផ្លាស់ទីរវាងសន្លឹកនីមួយៗដោយមិនចាំបាច់ពឹងផ្អែកលើផ្ទាំងសន្លឹកស្តង់ដារនៃ Excel។

គម្រោងនេះបង្ហាញពីការអនុវត្តជាក់ស្តែងនៃការសម្អាតទិន្នន័យ ការបង្កើតគំរូទិន្នន័យ និងបច្ចេកទេសបញ្ញាអាជីវកម្ម (business intelligence) នៅក្នុងបរិបទស្ថាប័នពិតប្រាកដ និងគូសបញ្ជាក់ពីសារៈសំខាន់នៃការ ត្រួតពិនិត្យសុពលភាពទិន្នន័យ នៅពេលធ្វើការជាមួយប្រភពទិន្នន័យជាក់ស្តែង ដែលមានរចនាសម្ព័ន្ធមិនស៊ីសង្វាក់គ្នា។

ABSTRACT

This report presents the work completed during a Data Analyst internship at MEATOPHUM Microfinance, a financial institution based in Phnom Penh, Cambodia, under the supervision of the Head of Finance. The internship focused on building an automated Income and Expense dashboard to support monthly financial reporting across the institution's 19 branches.

Prior to this project, monthly income and expense reports were compiled manually: figures from each branch's raw export were retyped into a summary sheet every month, a process that was time-consuming and prone to human error. The objective of this project was to replace this manual workflow with a structured, self-updating Excel dashboard that could be refreshed with a single click whenever new monthly data became available.

To achieve this, a Python-based data cleaning pipeline was developed to extract and reshape the raw monthly export into a clean, structured table. During development, a data quality issue was identified and resolved: a stray, undated table embedded in the source file was causing certain expense figures to be duplicated; a validation check was added to the pipeline to automatically detect and exclude such anomalies. The cleaned data was then loaded into Excel using Power Query and organized into a star schema data model, consisting of one fact table and three dimension tables (Branch, Date, and Account). A Conditional Column using keyword matching was implemented in Power Query to group over sixty raw expense line items into twelve interpretable categories for reporting purposes. Data Analysis Expressions (DAX) measures were then created to calculate key financial indicators, including Total Income, Total Expense, Net Income, and Profit Margin.

The final workbook is structured into three main sheets: Analysis, Dashboard, and Summary. The Dashboard provides interactive Pivot Charts and Key Performance Indicator (KPI) cards. To enhance usability, clickable image-based buttons were added using VBA macros and hyperlinks, allowing the Head of Finance to refresh data and navigate seamlessly between sheets without relying on standard Excel tabs. This project demonstrates the practical application of data cleaning, data modeling, and business intelligence techniques within a real organizational setting, and highlights the importance of data validation when working with real-world, inconsistently structured data sources.

LIST OF ABBREVIATIONS

API	Application Programming Interface
BI	Business Intelligence
CEO	Chief Executive Officer
DAX	Data Analysis Expressions
ETL	Extract, Transform, and Load
GL	General Ledger
GUI	Graphical User Interface
HR	Human Resources
ISO	International Organization for Standardization
ITC	Institute of Technology of Cambodia
KHR	Cambodian Riel
KPI	Key Performance Indicator
MIS	Management Information System
NPL	Non-Performing Loan
SME	Small and Medium-sized Enterprise
SQL	Structured Query Language
UI	User Interface
VBA	Visual Basic for Applications

SUMMARY

This report summarizes a Data Analyst internship completed at MEATOPHUM Microfinance in Phnom Penh, Cambodia, under the supervision of the Head of Finance. The project's goal was to replace a manual, error-prone monthly income and expense reporting process across the institution's 19 branches with an automated, self-refreshing Excel dashboard.

A Python-based pipeline was built to clean and reshape the raw monthly export, including a fix for a data quality issue that was causing certain expense figures to be duplicated. The cleaned data was structured in Excel using Power Query and a star schema data model (one fact table and three dimension tables), with DAX measures calculating Total Income, Total Expense, Net Income, and Profit Margin. A Conditional Column with keyword matching was used to group over sixty raw expense categories into twelve clear reporting groups.

The final workbook includes an Analysis sheet, a Dashboard sheet with interactive Pivot Charts and KPI cards, and a Summary sheet, with clickable VBA-powered navigation buttons for ease of use. The project demonstrates practical application of data cleaning, data modeling, and dashboard design in a real business setting.

TABLE OF CONTENTS

ACKNOWLEDGEMENT **1**

អត្ថបទសេចក្តីថ្លែងអំណរគុណ **2**

ABSTRACT **4**

LIST OF ABBREVIATIONS **5**

SUMMARY **6**

TABLE OF CONTENTS **7**

LIST OF FIGURES **10**

LIST OF TABLES **11**

CHAPTER 1 INTRODUCTION **12**

1.1 Presentation of Internship 12

1.1.1 Objective of Internship 12

1.1.2 Duration of Internship 12

1.2 Presentation of Organization 12

1.2.1 General Information of Company 12

1.2.2 Services of Company 13

1.2.3 Vision & Mission 13

1.2.4 Organization Chart 13

1.2.5 Address & Contact 14

CHAPTER 2 PRESENTATION OF THE PROJECT **15**

2.1 General Presentation of Project 15

2.2. Problematic 15

2.3 Objectives 15

2.4 Planning of Project 16

CHAPTER 3 LITERATURE REVIEW **18**

3.1. Manual Spreadsheet-Based Reporting and Its Risks 18

3.2. Business Intelligence Tools for Financial Reporting 18

3.2.1. Microsoft Power BI 18

3.2.2. Excel-Based Self-Service BI (Power Query, Power Pivot, DAX) 18

3.3. Dimensional Modeling (Star Schema) 18

3.4. Workbook Automation with VBA 19

3.5. Research Gap and Unique Contribution of This Project 19

3.6. Summary of the Literature Review 19

CHAPTER 4 TOOL AND TECHNOLOGY REQUIREMENTS **21**

4.1. Functional Requirements 21

4.2. Non-functional Requirements 21
4.3. Tools and Technology Requirements 22
4.3.1. Python 22
4.3.2. Jupyter Notebook 22
4.3.3. Microsoft Excel 22
4.3.4. Power Query 23
4.3.5. Power Pivot and DAX 23
4.3.6. Visual Basic for Applications (VBA) 23
CHAPTER 5 SYSTEM ARCHITECTURE AND CONCEPTS 24
5.1 Python and the Pandas Library 24
5.1.1 Technology Overview 24
5.1.2 Data Extraction and Dynamic Header Detection 24
5.1.3 Custom Validation and Reshaping 24
5.1.4 Pipeline Architecture and Rationale 25
5.2 Power Query Data Transformation 25
5.2.1 Technology Overview 25
5.2.2 Automated Folder Consolidation 25
5.2.3 Expense Categorization Logic 25
5.2.4 Query Dependency Architecture 25
5.3 Star Schema Data Model (Power Pivot) 26
5.3.1 Conceptual Design 26
5.3.2 Table Structures and Relationships 26
5.3.3 Rationale for Dimensional Modeling 27
5.4 DAX (Data Analysis Expressions) 27
5.4.1 Measure Definitions 27
5.4.2 Financial Key Performance Indicators 27
5.4.3 Contextual Recalculation 28
5.5 Visual Basic for Applications (VBA) 28
5.5.1 Automation Objective 28
5.5.2 Execution Process and UI Integration 28
CHAPTER 6 IMPLEMENTATION 29
6.1 Project Topology and End-to-End Data Flow 29
6.1.1 System Topology Overview 29
6.1.2 File Inventory and Staging Folder Structure 30
6.2 Step-by-Step Pipeline Implementation 32
6.2.1 One-Time Historical Data Cleaning in Python (Transform_to_raw_data.ipynb) 32

6.2.1.1 Raw Data Ingestion and Empty Column Removal 32
6.2.1.2 Date Banner Extraction and Normalization 33
6.2.1.3 Date-Proximity Safety Check and Valid Block Filtering 33
6.2.1.4 Unpivoting (pd.melt), Account Classification, and Master File Export 35
6.2.2 Automated Folder Consolidation and Categorization in Power Query 37
6.2.2.1 Folder Connector Setup (Dashboard_Data) 37
6.2.2.2 Conditional Column Implementation for the 12 Expense Groups 38
6.2.2.3 Reference Query Chain for Star Schema Tables 39
6.2.3 Star Schema Modeling and DAX Programming in Power Pivot 40
6.2.3.1 Data Model Loading and Relationship Configuration 40
6.2.3.2 DAX Measure Implementation 40
6.3 Workbook Architecture and Dashboard Construction 41
6.3.1 Analytical Staging Worksheet (Analysis Sheet) 41
6.3.2 Interactive Executive Dashboard (Dashboard Sheet) 42
6.3.2.1 KPI Cards and PivotChart Layout 42
6.3.2.2 Slicers and Multi-Sheet Report Connections 43
6.3.3 Branch-Level Financial Statement (Summary / SUM-Auto Sheet) 44
6.4 Final Configuration and VBA Automation 45
6.4.1 VBA One-Click Refresh Implementation (RefreshDashboard) 45
6.4.2 Image-Based Navigation Buttons and Interface Polish 45
CHAPTER 7 RESULTS AND DISCUSSION 46
7.1 Data Cleaning and Validation Results 46
7.1.1 Detection and Exclusion of the Stray Undated Table 46
7.1.2 Consolidated Dataset Volume and Structure 46
7.2 Financial Performance Analysis (January–September 2026) 46
7.2.1 Longitudinal Monthly Performance 46
7.3 System Workflow Evaluation (Manual vs. Automated Process) 47
CHAPTER 8 CONCLUSION 49
8.1 Challenges 49
8.2 Experience and Lesson Learned 49
8.3 Further Work 50
REFERENCES 51
APPENDICES 52

LIST OF FIGURES

Figure 1.1: MEATOPHUM's Logo 13

Figure 1.2: Organization Chart 14

Figure 4.1: Python Logo 22

Figure 4.2: Jupyter Notebook Logo 22

Figure 4.3: Microsoft Excel logo 22

Figure 4.4: Power Query logo 23

Figure 4.5: Power Pivot & DAX logo 23

Figure 4.6: VBA logo 23

Figure 5.1: Screenshot of the Queries pane in Power Query 26

Figure 5.2: Screenshot of the Power Pivot relationship diagram 27

Figure 6.1: End-to-End Project Topology and Data Flow Diagram 30

Figure 6.2: Staging Folder Containing IS_Master_All.xlsx, August_2026_NewData_FlatFormat.xlsx, and September_2026_NewData_FlatFormat.xlsx 32

Figure 6.3: Python Code Snippet for Raw File Ingestion and Date Extraction in Transform_to_raw_data.ipynb 33

Figure 6.4: Python Code Snippet for the Date-Proximity Safety Check and Valid Block Filtering 35

Figure 6.5: Python Code Snippet for Unpivoting, Adding Category and Signed Amount, and Exporting IS_Master_All.xlsx 37

Figure 6.6: Power Query Editor Showing the Dashboard_Data Staging Query and Helper Queries 38

Figure 6.7: DAX Measures Defined in the Power Pivot Calculation Grid 41

Figure 6.8: Underlying PivotTables Structured on the Analysis Worksheet 42

Figure 6.9: Full View of the Completed Interactive Dashboard Worksheet 43

Figure 6.10: Interactive Branch Slicers with Report Connections Dialog 43

Figure 6.11: Interactive Month Slicers with Report Connections Dialog 44

Figure 6.12: Automated Branch-by-Branch Summary on the SUM-Auto Worksheet 45

LIST OF TABLES

Table 2.1: Planning of Project (weekly Gantt chart; week 1 starts on July 02, 2026).....16

Table 2.2: Project Phase Activities 17

Table 3.1: Summary of Related Works and Approaches 20

Table 6.1: Inventory of Project Files and Staging Folder Components.....32

Table 6.2: Sample Records Showing the Seven-Column Flat Schema of IS_Master_All.xlsx and Subsequent Monthly Exports 36

Table 6.3: Keyword-Matching Rules Grouping Raw Account Descriptions into Twelve Standardized Expense Categories 39

Table 7.1: Monthly Consolidated Income, Expense, Net Income, and Profit Margin (Jan–Sep 2026).....47

Table 7.2: Operational Comparison Between the Legacy Manual Workflow and the Automated Dashboard Solution 48

CHAPTER 1 INTRODUCTION

1.1 Presentation of Internship

This internship was completed as part of the 4th-year engineering internship requirement at the Institut de Technologie du Cambodge (ITC), within the Department of Applied Mathematics and Statistics, majoring in Data Science. The internship was carried out remotely at MEATOPHUM Microfinance, a microfinance institution based in Phnom Penh, Cambodia, over a period of three months from July 02, 2026, to October 02, 2026, under the supervision of Ms. Wong Somaly, Head of Finance.

During the internship, I worked in the role of Data Analyst Intern within the Finance Department. My main project consisted of designing and building an automated Income and Expense dashboard in Microsoft Excel, intended to replace the institution's existing manual monthly financial reporting process across its 19 branches. My responsibilities on this project included cleaning and validating raw monthly financial data using Python, designing a structured data model in Excel using Power Query and Power Pivot, calculating key financial indicators using DAX measures, and building an interactive dashboard with Pivot Charts, Key Performance Indicator (KPI) cards, and navigation features for ease of use by the Head of Finance.

1.1.1 Objective of Internship

This internship provided the opportunity to apply theoretical data science and analytics knowledge acquired during my studies to a real, practical business environment. It allowed me to gain hands-on experience in data cleaning, data modeling, and dashboard development using industry-standard tools such as Python, Power Query, and Power Pivot/DAX. Beyond technical skills, the internship also strengthened my problem-solving ability, attention to data quality, and communication skills through direct collaboration with my supervisor. Overall, this experience bridged the gap between academic learning and professional practice, and helped clarify my career interest in data analysis.

1.1.2 Duration of Internship

The duration of the internship is three months, starting from July 02, 2026, to October 02, 2026. As the internship was conducted remotely, the working schedule followed a flexible but regular arrangement, generally from 9:00 AM to 5:00 PM, Monday to Friday, with periodic check-in meetings and reporting to the supervisor to track progress.

1.2 Presentation of Organization

1.2.1 General Information of Company

MEATOPHUM Microfinance is a financial institution operating in Cambodia, dedicated to providing accessible financial services to local communities. Established to support grassroots economic development, the institution focuses on delivering reliable credit solutions to individuals and small businesses to improve their livelihoods. Operating across a network of 19 branches, MEATOPHUM Microfinance plays an important role in facilitating financial inclusion and economic empowerment throughout the country.

1.2.2 Services of Company

MEATOPHUM Microfinance primarily specializes in providing comprehensive loan services designed to meet the financial needs of its clients. These credit services support various purposes, empowering individuals and small-to-medium enterprises (SMEs) to expand their businesses, manage their operations, and improve their overall standard of living.



Figure 1.1: MEATOPHUM's Logo

1.2.3 Vision & Mission

Vision: To be a leading and trusted microfinance institution in Cambodia, recognized for empowering local communities, fostering sustainable economic growth, and improving the livelihoods of the Cambodian people.

Mission: To provide accessible, transparent, and highly responsive loan services to individuals and small enterprises, ensuring financial inclusion and delivering reliable support to grassroots economic development.

1.2.4 Organization Chart

The management structure of MEATOPHUM Microfinance for the year 2025 is overseen by a Board of Directors, who guide the Chief Executive Officer (CEO). The executive operations are divided into four main functional departments, alongside an independent audit unit (Figure 1.2):

- Senior Audit Unit: operates independently under the CEO, led by the TBR Audit Officer.
- Head of Credit: oversees the Credit Supporting Manager (responsible for Loan Sales, Quality Control, and the Recruiting/Training Unit) and the Credit Supporting/NPL Manager (responsible for Loan Solving NPL and Branch Set Up).
- Head of HR/Admin: manages the Admin/Purchase Unit and the HR/Personnel Unit.
- Head of Finance/Accounts: the department where the internship took place. It governs the MIS Unit and the Finance Unit, with the latter further divided into Accounting Supporting and Tax and Treasury and Bank.

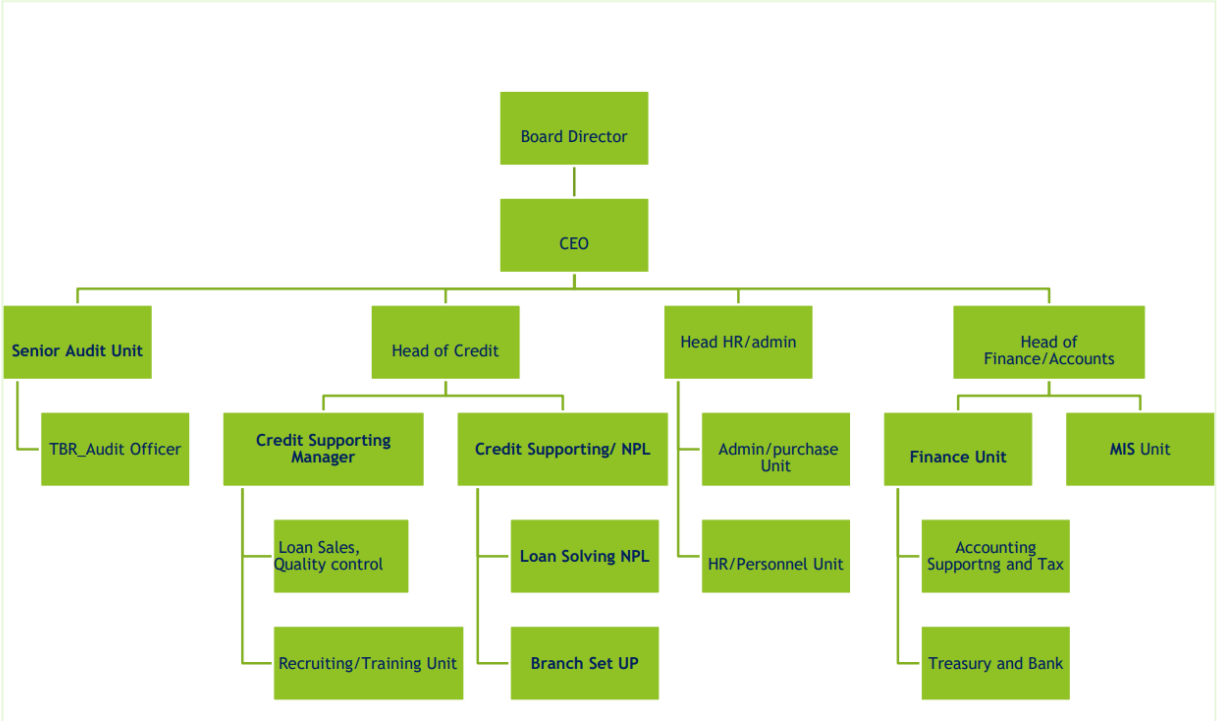


Figure 1.2: Organization Chart

1.2.5 Address & Contact

The headquarters of MEATOPHUM Microfinance is located at:

- Address: House No. 52B, Concrete Road, Phum Chrouy Basak 1, Sangkat Prek Pra, Khan Chbar Ampov, Phnom Penh, Cambodia.
- Contact number: +855 85 537 777

CHAPTER 2 PRESENTATION OF THE PROJECT

2.1 General Presentation of Project

The project undertaken during this internship consists of designing and building an automated Income and Expense Dashboard for MEATOPHUM Microfinance using Microsoft Excel. The dashboard consolidates monthly financial data — income and expense figures — from the institution's 19 branches into a single, centralized workbook. It combines a Python-based data cleaning pipeline with Excel's Power Query, Power Pivot, and DAX (Data Analysis Expressions) capabilities to transform raw, manually maintained monthly exports into a structured, interactive reporting tool. The final deliverable allows the Head of Finance to view Key Performance Indicators (KPIs), branch-level comparisons, expense category breakdowns, and month-over-month trends, all updatable through a single "Refresh All" action.

2.2. Problematic

Prior to this project, MEATOPHUM's monthly income and expense reporting process was entirely manual. Each month, financial figures exported from the accounting system had to be manually reviewed and retyped into a summary sheet, branch by branch, to produce a consolidated view for the Head of Finance. This manual process presented several issues:

- It was time-consuming, requiring repeated manual data entry every month across 19 branches.
- It was prone to human error, since figures were copied and retyped by hand rather than calculated automatically.
- The raw source data itself contained inconsistencies, such as a stray, undated table embedded within one month's data that, if not detected, would silently cause certain expense figures to be duplicated.
- There was no interactive or visual way to analyze the data — the existing summary only showed static totals, with no ability to filter by branch, month, or expense category.

2.3 Objectives

The main objective of this project was to design and implement an automated, self-refreshing Excel dashboard to replace the manual reporting process described above. More specifically, the project aimed to:

- Automate the monthly data cleaning process using Python, including detection and removal of data quality issues such as duplicate or stray records.
- Build a structured data model in Excel (a star schema with one fact table and three dimension tables) to support flexible, scalable analysis.
- Calculate key financial indicators — Total Income, Total Expense, Net Income, and Profit Margin — using DAX measures.
- Group over sixty raw expense line items into a smaller, interpretable set of twelve categories to support clearer reporting.

- Present the results through an interactive dashboard with KPI cards, Pivot Charts, and slicers, allowing the Head of Finance to explore the data by branch, month, and category without any manual data entry.
- Keep the entire solution within Microsoft Excel, without requiring Power BI or any additional licensed software, in line with the supervisor's requirement.

2.4 Planning of Project

The project was carried out over the three-month internship period, from July 02, 2026, to October 02, 2026, following the phases below:

Activities	Week												
	1	2	3	4	5	6	7	8	9	10	11	12	13
Onboarding & Requirements Gathering													
Data Cleaning Pipeline (Python)													
Data Modeling (Power Query & Star Schema)													
Measures & Categorization (DAX)													
Dashboard Development													
Visual Design & Automation (VBA)													

Table 2.1: Planning of Project (weekly Gantt chart; week 1 starts on July 02, 2026)

Phase	Period	Activities
1. Onboarding & Requirements Gathering	Jul 02 – Jul 15, 2026	Understood the existing manual reporting process; reviewed raw monthly export files (IS-All, SUM-IS sheets); identified reporting requirements with the supervisor.

Phase	Period	Activities
2. Data Cleaning Pipeline (Python)	Jul 16 – Aug 05, 2026	Built the Python pipeline to extract, unpivot, and clean the monthly export; identified and fixed a duplicate-row data quality issue.
3. Data Modeling (Power Query & Star Schema)	Aug 06 – Aug 20, 2026	Loaded cleaned data into Excel via Power Query; designed the star schema (Fact_Table, Dim_Branch, Dim_Date, Dim_Account).
4.Measures & Categorization (DAX)	Aug 21 – Sep 05, 2026	Created DAX measures (Total Income, Total Expense, Net Income, Profit Margin); built the Conditional Column to group expense categories.
5. Dashboard Development	Sep 06 – Sep 20, 2026	Built Pivot Tables, Pivot Charts, KPI cards, slicers, and the Analysis/Dashboard/Summary sheet structure.
6. Visual Design & Automation	Sep 21 – Sep 27, 2026	Applied the dashboard's visual design; added VBA macro-powered Refresh button and navigation buttons.
7. Testing, Review & Documentation	Sep 28 – Oct 02, 2026	Verified data accuracy against source; supervisor review; prepared internship documentation and thesis report.

Table 2.2: Project Phase Activities

CHAPTER 3 LITERATURE REVIEW

Before designing the dashboard, existing approaches to financial reporting automation were reviewed, both within MEATOPHUM's own workflow and in common industry practice. This chapter is organized by theme: the limits of manual spreadsheet reporting, the business intelligence tools available, the data modeling approach, workbook automation, and the gap this project addresses. Table 3.1 at the end of the chapter summarizes the works and approaches reviewed.

3.1. Manual Spreadsheet-Based Reporting and Its Risks

MEATOPHUM's existing monthly reporting process relied on manually retyping branch-level totals into a summary worksheet (SUM-IS) each month. This kind of manual consolidation is common in organizations that have not yet adopted dedicated business intelligence tools, and its risks are well documented. Panko (2016), reviewing research on spreadsheet errors, reports that errors are rare on a per-cell basis, but that in large spreadsheets at least one incorrect bottom-line value is very likely to be present. He also reports that such errors are extremely difficult to detect and correct, and that spreadsheet developers and organizations tend to be overconfident in the accuracy of their spreadsheets. At MEATOPHUM, where totals from 19 branches were retyped every month, a single transcription mistake could remain unnoticed in the consolidated report. This motivated two choices in this project: removing manual re-entry through automated data loading, and verifying calculated totals against the source data during development.

3.2. Business Intelligence Tools for Financial Reporting

3.2.1. Microsoft Power BI

A common industry approach to automating this kind of reporting is Microsoft Power BI, which offers data modeling and DAX-based calculation capabilities similar to those used in this project. However, Power BI is a separate application from Excel, the tool the Head of Finance already uses daily, and adopting it may involve additional software and licensing considerations depending on how reports are shared. This did not align with the supervisor's requirement to keep the final deliverable within Microsoft Excel, so Power BI was not selected.

3.2.2. Excel-Based Self-Service BI (Power Query, Power Pivot, DAX)

As an alternative, Microsoft Excel's built-in Power Query, Power Pivot, and DAX features provide comparable data transformation, modeling, and calculation capabilities directly inside the familiar Excel interface (Microsoft, n.d.-a, n.d.-c). Power Query loads and transforms data through a recorded list of steps that can be reviewed and edited. DAX defines measures that recalculate automatically according to whichever filters are active. This approach was chosen because it allowed the automation of data cleaning and calculation while keeping the final file in a format the supervisor could open, refresh, and use without additional training or software licensing.

3.3. Dimensional Modeling (Star Schema)

To structure the underlying data model, this project adopted the star schema approach to dimensional modeling described by Kimball and Ross (2013). It separates a central fact table (containing measurable transaction data) from surrounding dimension tables (containing descriptive attributes such as branch, date, and account). Microsoft's modeling guidance describes

the star schema as a mature approach widely adopted by relational data warehouses, and notes that its principles are highly relevant to building models optimized for performance and usability (Microsoft, n.d.-d). Although that guidance is written for Power BI, it describes the same relationship-based, measure-driven modeling approach used in this project's Excel data model. A star schema was selected over a single flat table because it improves scalability. For example, new months or branches can be added simply by inserting new rows, without rebuilding formulas or relationships.

3.4. Workbook Automation with VBA

To further reduce manual effort for the end user, this project also made use of Visual Basic for Applications (VBA), Microsoft Office's built-in automation language. VBA allows repetitive actions, such as refreshing every query and Pivot Table in a workbook, to be triggered with a single click rather than performed manually through multiple menus (Microsoft, n.d.-b). A simple macro was written and assigned to a clickable image button on the Dashboard sheet, allowing the Head of Finance to refresh the entire dashboard without needing any knowledge of Power Query or VBA herself.

3.5. Research Gap and Unique Contribution of This Project

While the general techniques used (Power Query, DAX, star schema modeling, VBA) are well established, their combined application here was specifically adapted to MEATOPHUM's real, inconsistent data source. In particular, the data cleaning pipeline includes a custom validation step, developed during this project, to automatically detect and exclude a stray, undated table found embedded within the raw monthly export. The sources reviewed in this chapter do not address this kind of issue, since they generally focus on well-structured input data.

3.6. Summary of the Literature Review

In conclusion, the sources reviewed show three things:

- Manual spreadsheet-based reporting carries a well-documented risk of undetected errors (Panko, 2016).
- Excel's built-in Power Query, Power Pivot, and DAX features, combined with a star schema model (Kimball & Ross, 2013; Microsoft, n.d.-d), can deliver a business-intelligence-style solution without additional tooling.
- VBA can reduce the end user's effort to a single action (Microsoft, n.d.-b).

What these sources do not address is how to handle raw source data that is itself inconsistently structured. This project fills that gap through its custom validation step. Table 3.1 summarizes the works and approaches reviewed in this chapter.

Author(s)	Approach	Context / Domain	Key Contribution

Panko (2016)	Review of spreadsheet error research	End-user spreadsheets in organizations	Errors are rare per cell but likely in a large spreadsheet's bottom line; hard to detect; developers overconfident
Kimball & Ross (2013)	Dimensional modeling (star schema)	Data warehouse design	Separates a central fact table from descriptive dimension tables
Microsoft (n.d.-d)	Star schema modeling guidance	Power BI data models	Describes star schema as a mature approach relevant to model performance and usability
Microsoft (n.d.-c)	Power Query	Data import and transformation	Loads and transforms data through reviewable steps
Microsoft (n.d.-a)	DAX	Calculations in Power Pivot / Power BI models	Measures that recalculate with the active filters
Microsoft (n.d.-b)	VBA	Microsoft Office automation	Macros that automate repetitive actions such as Refresh All
This project	Python validation + Power Query + star schema + DAX + VBA, all in Excel	MEATOPHUM income and expense reporting	Handles an inconsistent raw source (stray undated table) and refreshes with one click

Table 3.1: Summary of Related Works and Approaches

CHAPTER 4 TOOL AND TECHNOLOGY REQUIREMENTS

4.1. Functional Requirements

The functional requirements define the specific behaviors, capabilities, and data processing tasks the automated dashboard must perform to fulfill the business needs of MEATOPHUM Microfinance.

- **Data Cleaning and Validation:** The system must automatically extract, unpivot, and clean the raw monthly income and expense exports. Crucially, it must detect and exclude invalid or stray data blocks (such as the undated tables identified during development) before they enter the data model.
- **Data Consolidation:** The system must seamlessly combine income and expense data from all 19 branches into a single, unified dataset.
- **Expense Categorization:** The solution must group over sixty raw expense line items into a condensed set of twelve interpretable categories to facilitate clearer reporting.
- **Financial KPI Calculation:** The system must automatically calculate core financial metrics, including Total Income, Total Expense, Net Income, and Profit Margin, utilizing DAX measures.
- **Interactive Filtering:** The dashboard must allow the end user to dynamically filter all visualizations and metrics by branch and by month using slicers.
- **Data Visualization:** The system must present the financial data through intuitive Key Performance Indicator (KPI) cards, trend line charts, expense breakdown charts, branch comparison charts, and growth percentage charts.
- **Branch-Level Summary Reporting:** The dashboard must faithfully reproduce the branch-by-branch income statement format familiar to the Head of Finance in an interactive, filterable table.
- **One-Click Refresh & Navigation:** The entire dashboard (queries, pivot tables, and charts) must update with a single action when new monthly data is added. Furthermore, clickable navigation buttons must be provided to seamlessly move between the Dashboard, Analysis, and Summary sheets.

4.2. Non-functional Requirements

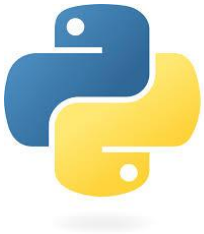
The non-functional requirements define the system's operational constraints, performance expectations, and usability standards.

- **Tool Constraint:** The solution must be built entirely within Microsoft Excel, without requiring Power BI or any additional licensed software, per the supervisor's explicit requirement.
- **Usability:** The dashboard must be highly accessible for a non-technical end user (the Head of Finance), requiring absolutely no knowledge of Python, Power Query, or DAX to operate.

- **Data Accuracy:** All calculated totals must match the original source data exactly, which was rigorously verified through manual cross-checking during the development phase.
- **Maintainability and Scalability:** The underlying star schema data model must support the addition of new months or branches simply by adding new rows of data, without requiring manual updates to existing formulas, table relationships, or dashboard layouts.
- **Reliability:** The Python data cleaning pipeline must be robust against inconsistencies in the raw source file—such as unexpected extra tables or formatting irregularities—ensuring that silently incorrect results are not produced.
- **Performance:** The data refresh process should execute efficiently, completing within a few seconds, even as historical monthly data accumulates in the system over time.

4.3. Tools and Technology Requirements

4.3.1. Python



Python, utilizing the pandas and numpy libraries, is used for extracting, cleaning, validating, and reshaping the raw monthly Excel export into a structured long-format table.

Figure 4.1: Python Logo

4.3.2. Jupyter Notebook



Jupyter Notebook serves as the primary development environment used to write, test, and document the Python data cleaning pipeline.

Figure 4.2: Jupyter Notebook Logo

4.3.3. Microsoft Excel



Microsoft Excel is the primary platform for the final deliverable, chosen per the supervisor's requirement to build the solution without Power BI.

Figure 4.3: Microsoft Excel logo

4.3.4. Power Query



Power Query, utilizing the M language, is responsible for loading the cleaned data into Excel, merging and categorizing expense groups using Conditional Columns, and combining monthly files via a Folder source.

Figure 4.4: Power Query logo

4.3.5. Power Pivot and DAX



Power Pivot is used for structuring the cleaned data into a star schema (one fact table and three dimension tables: Branch, Date, Account). Data Analysis Expressions (DAX) are utilized for calculating key financial measures: Total Income, Total Expense, Net Income, and Profit Margin.

Figure 4.5: Power Pivot & DAX logo

4.3.6. Visual Basic for Applications (VBA)



VBA is used for automating the "Refresh All" action through a macro assigned to a clickable image button to ensure ease of use for the final user.

Figure 4.6: VBA logo

CHAPTER 5 SYSTEM ARCHITECTURE AND CONCEPTS

This chapter details the conceptual framework and theoretical architecture of the data pipeline. It outlines the specific technologies utilized, their structural implementation, and the rationale behind their selection for automating the MEATOPHUM Microfinance reporting workflow.

5.1 Python and the Pandas Library

5.1.1 Technology Overview

Python is a general-purpose, high-level programming language widely used in data analysis and automation. For this project, the core framework relies on pandas, a powerful data manipulation library that provides specialized data structures—primarily the DataFrame, a tabular structure similar to a spreadsheet—and functions for reading, cleaning, reshaping, and writing complex tabular data.

5.1.2 Data Extraction and Dynamic Header Detection

The raw monthly export file contains income and expense data organized as a series of stacked "blocks," one per month. Each block possesses its own header row and a mix of text labels and numeric values arranged in a wide format (one column per branch). Because the sheet's structure does not follow a single consistent table layout, the pipeline reads the raw sheet into a pandas DataFrame with no assumed header row. The script dynamically locates each month's block by detecting rows containing a date label, and identifies each block's header row by detecting the text "GL Code".

5.1.3 Custom Validation and Reshaping

To ensure data integrity, a custom validation step evaluates each detected header row by checking that a date label exists within a small number of rows above it. Header rows without a nearby date label are strictly excluded. This targeted logic was implemented to handle a stray, undated table discovered in the raw source during development, which would have otherwise been misread as a valid month. Following validation, the pipeline unpivots the wide table into a long format, producing one row per transaction: Date, GL Code, Description, Branch, and Amount. Finally, it classifies each row as "Income" or "Expense" based on the first digit of its GL Code and computes a signed amount (with expenses stored as negative values) to support later aggregation.

5.1.4 Pipeline Architecture and Rationale

The pipeline runs as a standalone script (developed and tested in a Jupyter Notebook), entirely independent of Excel. It reads one Excel file as input and writes a clean Excel file as output, with no dependency on the internal structure of the destination workbook. Pandas was selected because the raw source data's inconsistent formatting and varying row counts are difficult to clean reliably using Excel formulas or Power Query alone. Python's explicit, inspectable logic enabled the creation of the date-proximity check, a critical validation rule that would be highly complex to debug in Power Query's M language.

5.2 Power Query Data Transformation

5.2.1 Technology Overview

Power Query is a data connection and transformation technology built directly into Microsoft Excel. It utilizes its own functional language, M, to import data from external sources, clean or reshape it, and load it into the workbook via repeatable, automated steps.

5.2.2 Automated Folder Consolidation

To handle ongoing reporting, a Folder source is configured to monitor a designated directory. It automatically combines every cleaned monthly file placed in this folder into a single staging query named Dashboard_Data. This ensures that adding a new month's file is sufficient to extend the dataset without manually modifying any underlying query.

5.2.3 Expense Categorization Logic

A Conditional Column utilizes keyword matching (the Power Query "contains" condition) on the Description field to categorize data. It assigns each of the sixty-plus raw expense line items to one of twelve broader Expense Group categories (e.g., grouping all salary- and bonus-related items under "Staff Cost"). These rules are strictly evaluated in order, from most specific to most general, ensuring that items matching multiple keywords are classified correctly. Every transformation is recorded as a named step in the query's Applied Steps list, ensuring complete transparency.

5.2.4 Query Dependency Architecture

The Power Query architecture forms a strict dependency chain. The Folder source feeds the staging query, which in turn feeds the Fact table query. The three dimension table queries (Dim_Date, Dim_Branch, Dim_Account) are built by referencing the Fact table query rather than the original source file. This guarantees that all four queries stay perfectly synchronized whenever new data is added to the folder. Power Query was chosen because it executes these transformations once at refresh time, eliminating manual data entry while utilizing native Excel tools at no additional cost.

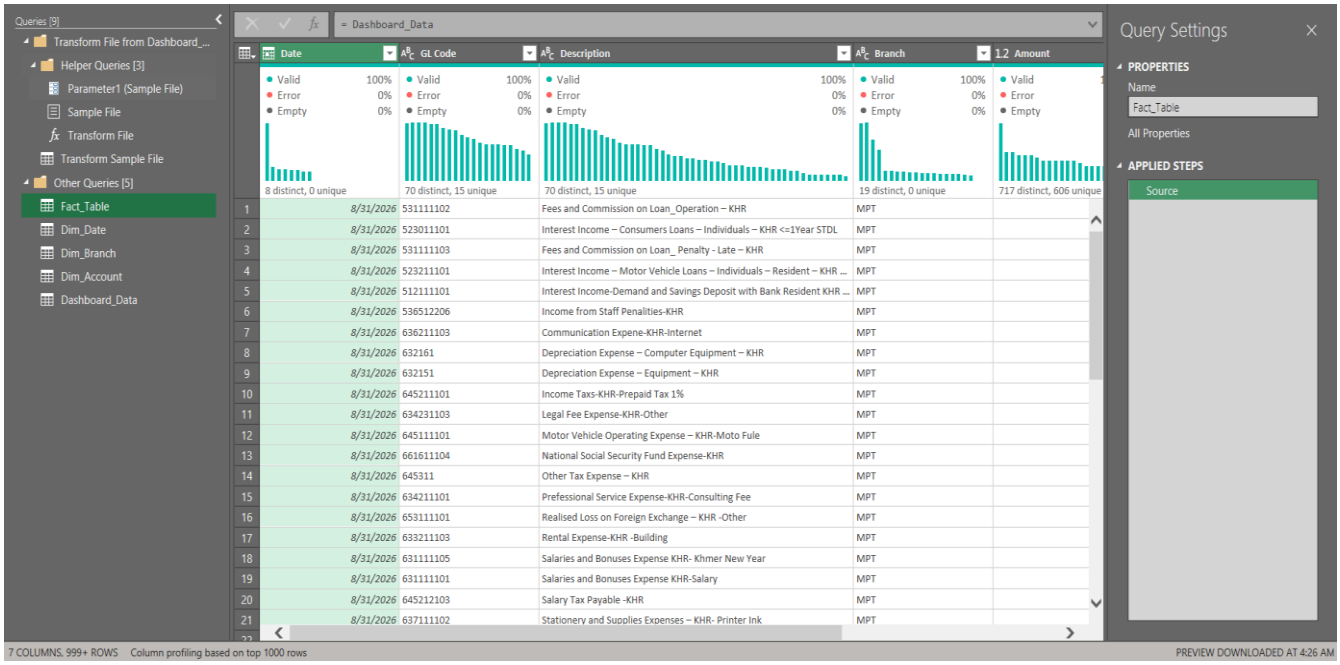


Figure 5.1: Screenshot of the Queries pane in Power Query

5.3 Star Schema Data Model (Power Pivot)

5.3.1 Conceptual Design

A star schema is a highly optimized data organization pattern utilized in data warehousing and business intelligence (Kimball & Ross, 2013). It separates a central fact table (holding measurable, transactional data) from surrounding dimension tables (holding descriptive attributes used to filter and group that data). In this project, the model is built and managed using Power Pivot, Excel's internal in-memory data modeling engine.

5.3.2 Table Structures and Relationships

The constructed model consists of four primary tables. The Fact_Table acts as the fact table, containing one row per transaction (Date, GL Code, Branch, and Amount). It is surrounded by three dimension tables: Dim_Date (containing derived calendar columns like Year and Quarter), Dim_Branch (one row per branch code), and Dim_Account (containing GL Codes, Descriptions, and Categories).

Each dimension table is connected to the Fact_Table through a one-to-many relationship (e.g., one branch can relate to many transactions), defined strictly in Power Pivot's relationship diagram. These relationships allow a single user filter—such as selecting one month in a slicer—to automatically filter every related table and subsequent PivotChart.

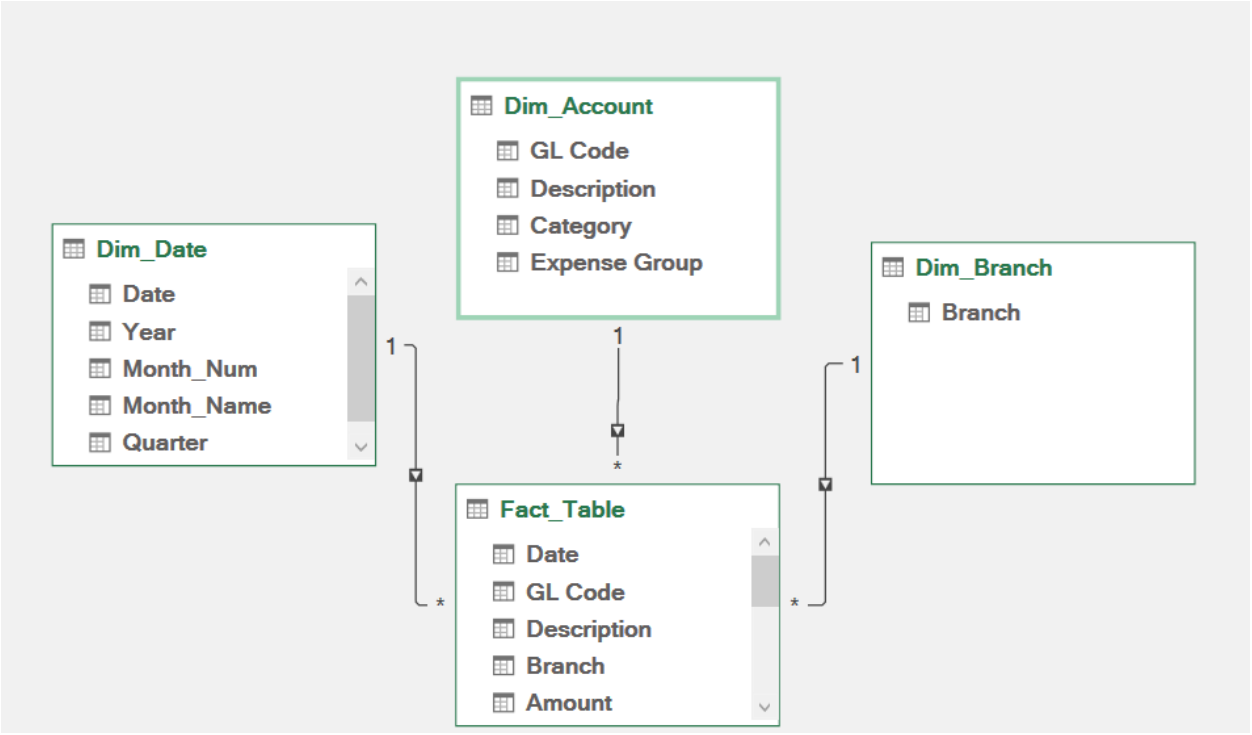


Figure 5.2: Screenshot of the Power Pivot relationship diagram

5.3.3 Rationale for Dimensional Modeling

This star structure was deliberately chosen over a single flat table to optimize performance. By isolating descriptive text (such as full account descriptions) into dimension tables, the model avoids repeating text strings across thousands of transaction rows, keeping the file size small and refresh speeds fast. Furthermore, it provides seamless scalability; extending the dataset with new branches or months simply adds rows to the `Fact_Table` without requiring existing relationships or charts to be rebuilt.

5.4 DAX (Data Analysis Expressions)

5.4.1 Measure Definitions

DAX is the functional formula language used by Power Pivot to define calculations, known as measures. Unlike standard spreadsheet formulas that operate on individual cells, DAX measures operate across the entire relational data model contextually.

5.4.2 Financial Key Performance Indicators

Four critical measures were developed for this reporting system:

- **Total Income:** `CALCULATE(SUM(Fact_Table[Amount]), Dim_Account[Category] = "Income")`
- **Total Expense:** `CALCULATE(SUM(Fact_Table[Amount]), Dim_Account[Category] = "Expense")`
- **Net Income:** `[Total Income] - [Total Expense]`

- **Profit Margin:** DIVIDE([Net Income], [Total Income])

5.4.3 Contextual Recalculation

These measures are defined once on the Fact_Table and are universally reusable across every PivotTable and PivotChart in the workbook. DAX was chosen over static cell formulas because measures recalculate automatically based on whatever filters are currently active (e.g., checking Profit Margin for a specific branch). This ensured that the exact same measures built for a January test case continued to work flawlessly when August and September data were introduced.

5.5 Visual Basic for Applications (VBA)

5.5.1 Automation Objective

VBA is Microsoft Office's native programming language designed for automating repetitive background tasks within the application environment.

5.5.2 Execution Process and UI Integration

A dedicated macro named RefreshDashboard was programmed to execute the ThisWorkbook.RefreshAll command. This single command systematically refreshes every Power Query connection, PivotTable, and PivotChart in the workbook simultaneously. The macro is stored in a standard module within a macro-enabled workbook (.xlsm) and is assigned directly to a graphical user interface (GUI) image placed on the Dashboard sheet. VBA was integrated to abstract the technical complexity of the refresh process away from the Head of Finance, reducing a multi-step navigation sequence into a single, intuitive button click.

CHAPTER 6 IMPLEMENTATION

This chapter presents the practical execution and technical realization of the automated Income and Expense reporting system developed for MEATOPHUM Microfinance. Building upon the conceptual framework established in Chapter 5, the implementation is structured into four primary phases: defining the end-to-end project topology, executing the multi-stage data engineering pipeline across Python, Power Query, and Power Pivot, constructing the three-sheet analytical workbook, and configuring the final Visual Basic for Applications (VBA) automation settings.

6.1 Project Topology and End-to-End Data Flow

6.1.1 System Topology Overview

The implemented system architecture separates one-time historical data cleaning from ongoing monthly reporting through a structured Extract, Transform, and Load (ETL) topology coupled with an in-memory dimensional data model inside Microsoft Excel. Historical financial data originates from the institution's legacy accounting workbook named 20260531-Income and Expensed-Jan to July 26.xlsx, which contains the stacked multi-month branch worksheet (IS-All) and the legacy manual summary worksheet (SUM-IS) covering January through July 2026. Rather than modifying the raw historical accounting export manually, the workflow passes the IS-All worksheet once through an external Python preprocessing script (Transform_to_raw_data.ipynb) executed in Jupyter Notebook.

Once the Python script extracts the dates, validates the monthly blocks, unpivots the 19 branch columns, and standardizes the records, it exports a unified seven-month master file named IS_Master_All.xlsx into a designated local staging folder (Dashboard_Data). For subsequent monthly cycles—demonstrated using the August and September 2026 datasets (August_2026_NewData_FlatFormat.xlsx and September_2026_NewData_FlatFormat.xlsx)—monthly data exported from the system in the standardized flat format is placed directly into the Dashboard_Data folder without requiring repeated Python execution. Microsoft Excel's Power Query engine monitors this directory via a Folder connection, automatically appending IS_Master_All.xlsx and all subsequent monthly flat files into a unified staging query (Dashboard_Data). From this staging layer, Power Query partitions the records into one transactional fact table (Fact_Table) and three descriptive dimension tables (Dim_Date, Dim_Branch, and Dim_Account), loading them directly into the Power Pivot Data Model. Finally, Data Analysis Expressions (DAX) measures and interactive PivotTables render the financial indicators across the Analysis, Dashboard, and Summary worksheets, while a VBA macro governs the one-click refresh lifecycle.

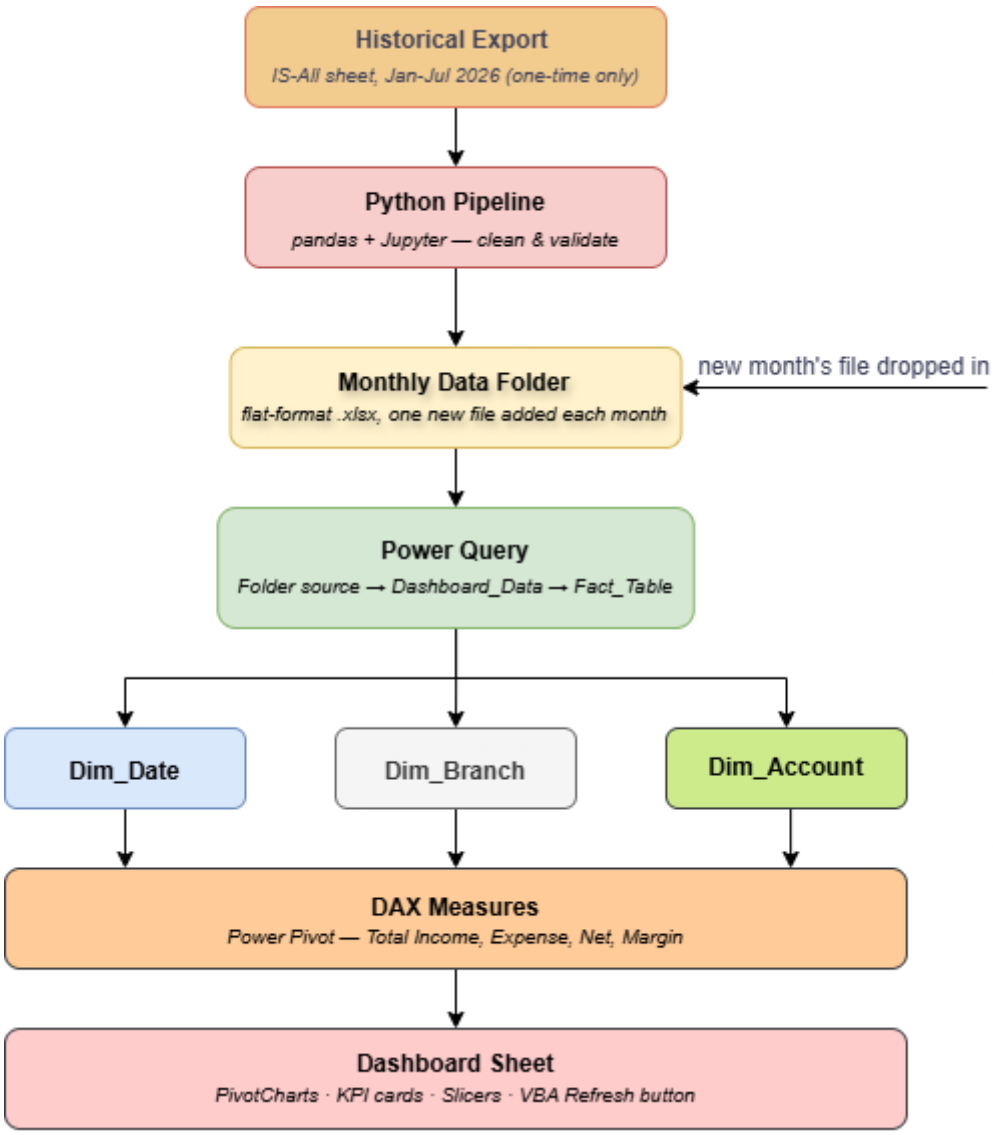


Figure 6.1: End-to-End Project Topology and Data Flow Diagram

6.1.2 File Inventory and Staging Folder Structure

To guarantee seamless automation for the Finance Department, the project workspace separates the historical raw export and Python transformation script from the active Dashboard_Data staging folder monitored by Power Query. All files placed inside the Dashboard_Data directory strictly follow the standardized seven-column flat schema (Date, GL Code, Description, Branch, Amount, Category, and Signed Amount). Table 6.1 details the inventory of files used in the implementation.

File or Folder Name	Type	Size	Role in System Implementation
20260531-Income and Expensed-Jan to July 26.xlsx	Excel Workbook (.xlsx)	-	**Raw Source:** Legacy multi-block accounting export

File or Folder Name	Type	Size	Role in System Implementation
			containing the IS-All and SUM-IS sheets for January–July 2026.
Transform_to_raw_data.ipynb	Jupyter Notebook (.ipynb)	-	**Script:** One-time Python preprocessing script that cleans, validates, and unpivots the historical IS-All sheet.
Dashboard_Data	Directory (Folder)	-	**Staging Folder:** Active directory monitored by Power Query's From Folder connector.
IS_Master_All.xlsx	Excel Workbook (.xlsx)	106 KB	Cleaned, unpivoted historical master table (Jan–Jul 2026) generated by Python and stored in the staging folder.
August_2026_NewData_FlatFormat.xlsx	Excel Workbook (.xlsx)	22 KB	Standardized flat-format monthly file for August 2026 placed directly into the staging folder.

File or Folder Name	Type	Size	Role in System Implementation
September_2026_NewData_FlatFormat.xlsx	Excel Workbook (.xlsx)	22 KB	Standardized flat-format monthly file for September 2026 placed directly into the staging folder.

Table 6.1: Inventory of Project Files and Staging Folder Components

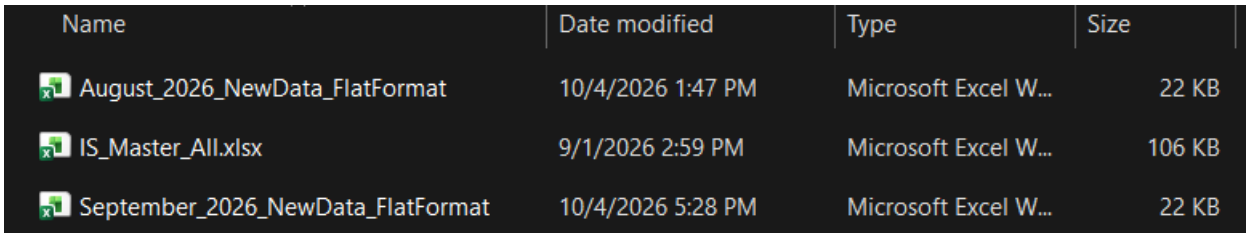


Figure 6.2: Staging Folder Containing IS_Master_All.xlsx, August_2026_NewData_FlatFormat.xlsx, and September_2026_NewData_FlatFormat.xlsx

6.2 Step-by-Step Pipeline Implementation

6.2.1 One-Time Historical Data Cleaning in Python (Transform_to_raw_data.ipynb)

6.2.1.1 Raw Data Ingestion and Empty Column Removal

The historical data remediation script was implemented in Transform_to_raw_data.ipynb using pandas and numpy. In the first stage of execution, the script loads the IS-All worksheet from 20260531-Income and Expensed-Jan to July 26.xlsx using pd.read_excel(..., sheet_name="IS-All", header=None). Setting header=None prevents pandas from misinterpreting the first row of metadata as column headers. Immediately after ingestion, df_raw.dropna(axis=1, how='all') removes completely blank spacer columns, and df_raw.columns = range(df_raw.shape[1]) resets the column indices to sequential integers starting at 0.

```python
import pandas as pd
import numpy as np

# ---------------------------------------------------------------------
# 1. Load the raw file
# ---------------------------------------------------------------------
file_path = r"D:\ITC\Intern Project\Data Analysis Project\Summary Data Automation\20260531-Income and Expensed-Jan to July 26.xlsx"
df_raw = pd.read_excel(file_path, sheet_name="IS-All", header=None)

df_raw = df_raw.dropna(axis=1, how='all')
df_raw.columns = range(df_raw.shape[1])

# ---------------------------------------------------------------------
# 2. Extract and clean the Dates (same as before)
# ---------------------------------------------------------------------
date_mask = df_raw[0].astype(str).str.startswith('Date', na=False)
df_raw['Date'] = np.where(date_mask, df_raw[0].astype(str), np.nan)
df_raw['Date'] = df_raw['Date'].ffill()
df_raw['Date'] = df_raw['Date'].astype(str).str.replace('Date :', '').str.strip()
df_raw['Date'] = df_raw['Date'].str.replace(r'(\d+)(st|nd|rd|th)', r'\1', regex=True)
df_raw['Date'] = pd.to_datetime(df_raw['Date'], format='mixed', errors='coerce')
```

Figure 6.3: Python Code Snippet for Raw File Ingestion and Date Extraction in Transform_to_raw_data.ipynb

6.2.1.2 Date Banner Extraction and Normalization

In the raw IS-All sheet, each monthly block is preceded by a text banner in Column 0 formatted with a "Date :" prefix and ordinal day suffixes (such as 31st, 30th, or 28th). To assign the correct reporting date to every transaction row within a block, the script constructs a boolean mask (date_mask = df_raw[0].astype(str).str.startswith('Date', na=False)) to isolate date banner rows and copies them into a new Date column, while setting non-banner rows to np.nan. Applying forward-fill (df_raw['Date'].ffill()) propagates each month's date label downward across all rows in its corresponding block. Next, string manipulation strips the "Date :" prefix, a regular expression (r'(\d+)(st|nd|rd|th)') removes ordinal suffixes from day numbers, and pd.to_datetime(..., format='mixed', errors='coerce') converts the cleaned strings into standardized ISO date objects (YYYY-MM-DD).

6.2.1.3 Date-Proximity Safety Check and Valid Block Filtering

During development, inspection of the IS-All sheet revealed a leftover utility notes table located below the July 2026 block that contained a "GL Code" header row but no date banner above it. Because forward-filling (ffill()) propagated July's date into this stray table, naive extraction caused specific July expense line items to be duplicated.

To resolve this issue programmatically, a two-part validation routine was implemented in Steps 3, 4, and 5 of the Python script:

1. **Header Proximity Verification:** The script collects all row indices where Column 0 equals 'GL Code' (header_rows) and all row indices where a date banner was detected (date_rows). For every header index h, the condition any(0 <= h - d <= 4 for d in date_rows) checks whether a valid 'Date :' label appears within 4 rows above the header. Headers satisfying this condition are appended to valid_header_rows, whereas headers lacking a nearby date banner are routed to skipped_header_rows and logged via a console warning.

2. **Block Cutpoint Masking and Numeric GL Filtering:** Column headers are assigned from the first validated block (valid_header_rows[0]), with the trailing column named 'Date'. The script then defines block boundaries (all_cutpoints = sorted(header_rows) + [len(df_raw)]) and constructs a boolean mask (valid_row_mask) that marks rows as True only if they fall between a validated header h and the next cutpoint. Combining valid_row_mask with a numeric check on the ledger code (pd.to_numeric(df_raw['GL Code'], errors='coerce').notna()) filters out all sub-headers, summary total rows, blank lines, and the entire skipped utility table after July.

```python
# ------------------------------------------------------------------
# 3. NEW SAFETY CHECK
#    Find every 'GL Code' header row. A REAL month-block header always
#    has a 'Date :' label a few rows above it. If a 'GL Code' header
#    has NO Date label nearby, it's a stray/leftover table (like the
#    utility notes table we found after July) -> we skip it.
# ------------------------------------------------------------------
header_rows = df_raw.index[df_raw[0] == 'GL Code'].tolist()
date_rows = df_raw.index[date_mask].tolist()

valid_header_rows = []
skipped_header_rows = []
for h in header_rows:
    has_date_nearby = any(0 <= h - d <= 4 for d in date_rows)
    (valid_header_rows if has_date_nearby else skipped_header_rows).append(h)

if skipped_header_rows:
    print(f"⚠️  Skipped {len(skipped_header_rows)} extra table(s) at row(s) {skipped_header_rows}.")
    print("    These have GL Code headers but NO 'Date :' label above them -> ")
    print("    treated as leftover notes, not real monthly data.")

if not valid_header_rows:
    raise ValueError("No valid month blocks found - check the file format!")

# ------------------------------------------------------------------
# 4. Apply the correct headers (from the first valid block)
# ------------------------------------------------------------------
headers = df_raw.iloc[valid_header_rows[0]].tolist()
headers[-1] = 'Date'
df_raw.columns = headers

# ------------------------------------------------------------------
# 5. Filter: only numeric GL Code rows, AND only rows inside a
#    VALID block (this is what blocks the duplicate rows)
# ------------------------------------------------------------------
is_numeric_gl = pd.to_numeric(df_raw['GL Code'], errors='coerce').notna()

all_cutpoints = sorted(header_rows) + [len(df_raw)]
valid_row_mask = pd.Series(False, index=df_raw.index)
for h in valid_header_rows:
    end = all_cutpoints[all_cutpoints.index(h) + 1]
    valid_row_mask.loc[h:end - 1] = True

df_clean = df_raw[is_numeric_gl & valid_row_mask].copy()
```

Figure 6.4: Python Code Snippet for the Date-Proximity Safety Check and Valid Block Filtering

6.2.1.4 Unpivoting (pd.melt), Account Classification, and Master File Export

In Step 6 of the script, the wide branch columns in df_clean are reshaped into a normalized long table using pd.melt(). The identifier columns are defined as id_cols = ['Date', 'GL Code', 'Description'], while branch_cols dynamically selects all 19 branch code columns (MPT, CHB, SNK, STR, KPC, SNL, PTR, STG, RTK, BKV, BLV, KHR 168, ASL, 999 Cash, KTL, SLG, SST, ASY168, and HO) by excluding id_cols, null column headers, and any column containing the word 'Total'.

In Step 7, after dropping null amounts (dropna(subset=['Amount'])), vectorized np.where() operations generate two derived analytical columns:

- **Category:** Evaluates df_melted['GL Code'].astype(str).str.startswith('5'), assigning 'Income' to ledger codes starting with 5 and 'Expense' to ledger codes starting with 6.
- **Signed Amount:** Assigns -df_melted['Amount'] when Category == 'Expense' and retains the positive Amount when Category == 'Income'.

Finally, Step 8 exports the resulting DataFrame to IS_Master_All.xlsx (index=False), producing the seven-column production baseline table shown in Table 6.2.

Date	GL Code	Description	Branch	Amount	Category	Signed Amount
2026-09-30	531111102	Fees and Commission on Loan_Operation – KHR	MPT	6,676,000	Income	6,676,000
2026-09-30	523011101	Interest Income – Consumers Loans – Individuals – KHR <=1Year STDL	MPT	522,500	Income	522,500
2026-09-30	636211103	Communication Expene-KHR-Internet	MPT	47,200	Expense	-47,200
2026-09-30	632161	Depreciation Expense – Computer Equipment – KHR	MPT	32,300	Expense	-32,300
2026-09-30	645211101	Income Taxs-KHR-Prepaid Tax 1%	MPT	55,200	Expense	-55,200

Table 6.2: Sample Records Showing the Seven-Column Flat Schema of IS_Master_All.xlsx and Subsequent Monthly Exports

```
# ----------------------------------------------------------------------
# 6. Melt / Unpivot the Data (same as before)
# ----------------------------------------------------------------------
id_cols = ['Date', 'GL Code', 'Description']
branch_cols = [c for c in df_clean.columns if c not in id_cols and pd.notna(c) and 'Total' not in
str(c)]
df_melted = pd.melt(df_clean, id_vars=id_cols, value_vars=branch_cols, var_name='Branch',
value_name='Amount')
df_melted = df_melted.dropna(subset=['Amount'])

# ----------------------------------------------------------------------
# 7. Add Category and Signed Amount (same as before)
# ----------------------------------------------------------------------
df_melted['Category'] = np.where(df_melted['GL Code'].astype(str).str.startswith('5'), 'Income',
'Expense')
df_melted['Signed Amount'] = np.where(df_melted['Category'] == 'Expense', -df_melted['Amount'],
df_melted['Amount'])

# ----------------------------------------------------------------------
# 8. Save
# ----------------------------------------------------------------------
df_melted.to_excel("IS_Master_All.xlsx", index=False)
print(f"Rows in final table: {len(df_melted)}")
print("Data is 100% production-ready!")
```

Figure 6.5: Python Code Snippet for Unpivoting, Adding Category and Signed Amount, and Exporting IS_Master_All.xlsx

6.2.2 Automated Folder Consolidation and Categorization in Power Query

6.2.2.1 Folder Connector Setup (Dashboard_Data)

To automate ongoing monthly data integration without re-running Python each month, Power Query was configured using Data -> Get Data -> From File -> From Folder directed at the Dashboard_Data folder. Power Query automatically generated the helper query pipeline (Parameter1, Sample File, Transform Sample File, and Transform File from Dashboard_Data) to read Sheet1 from every workbook in the directory and append all rows into the master staging query named Dashboard_Data. Whenever a new monthly file formatted in the same seven-column structure—such as August_2026_NewData_FlatFormat.xlsx or September_2026_NewData_FlatFormat.xlsx (which adds 430 rows covering 64 unique GL Codes across the 19 branches)—is dropped into the folder, Power Query consolidates it automatically upon refresh.

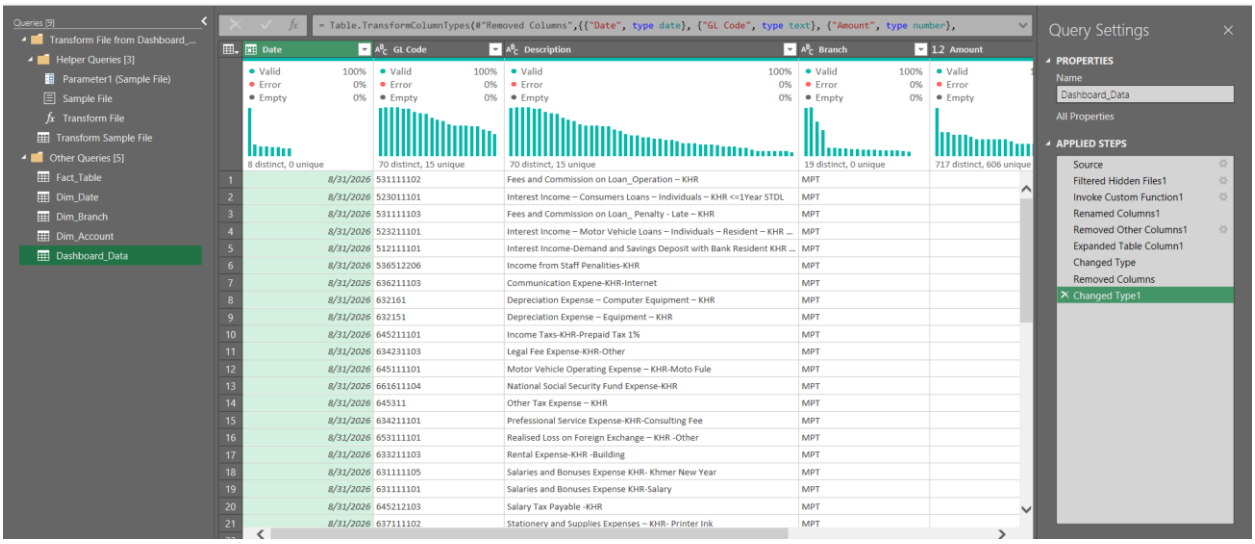


Figure 6.6: Power Query Editor Showing the Dashboard_Data Staging Query and Helper Queries

6.2.2.2 Conditional Column Implementation for the 12 Expense Groups

Across the unique GL Code line items present in MEATOPHUM Microfinance's chart of accounts, over sixty descriptions represent granular operating expenses. To summarize these items into executive-ready reporting categories, a Conditional Column named Expense Group was implemented inside the Dim_Account query in Power Query using sequential Text.Contains keyword matching on the Description column. Evaluated strictly in order, accounts where Category equals "Income" are assigned to "Income", while all expense descriptions are grouped into twelve standardized categories (eleven specific operational cost groups plus an "Other" fallback group), as shown in Table 6.3.

Rule Order	Keyword Matching Condition on Description	Assigned Expense Group	Total Expense (Jan–Sep 2026, KHR)
1	"Utility"	Utility	8,648,347
2	"Salary Tax", "Tax", "License Fee"	Tax & Statutory	7,703,557
3	"Rent"	Rent	99,635,900
4	"Depreciation", "Amortisation"	Depreciation & Amortisation	57,291,895
5	"Repair and Maintenance"	Repair & Maintenance	4,537,896

Rule Order	Keyword Matching Condition on Description	Assigned Expense Group	Total Expense (Jan–Sep 2026, KHR)
6	"Travel", "Motor Vehicle"	Travel & Vehicle	162,660,521
7	"Stationery", "Printing", "Communication"	Office & Admin	22,690,731
8	"Professional Service", "Legal Fee", "Prefessional"	Professional Service	27,871,101
9	"Interest Expense"	Interest Expense	150,000
10	"Provision Expense"	Loan Loss Provision	285,469,100
11	"Salar", "National Social Security", "Employee"	Staff Cost	1,348,224,423
12	Fallback (all remaining expense descriptions)	Other	412,534,471

Table 6.3: Keyword-Matching Rules Grouping Raw Account Descriptions into Twelve Standardized Expense Categories

6.2.2.3 Reference Query Chain for Star Schema Tables

Following the query dependency architecture established in Section 5.2.4, four star schema queries were constructed by referencing Dashboard_Data rather than re-reading the disk directory:

1. Fact_Table: References Dashboard_Data directly and retains all 3,664 consolidated transaction rows across January to September 2026.
2. Dim_Date: References Fact_Table, keeps only the Date column, removes duplicate dates, and extracts Year, Month_Num, Month_Name, and Quarter.
3. Dim_Branch: References Fact_Table, keeps the Branch column, and removes duplicates to isolate the 19 unique branch codes (MPT, CHB, SNK, STR, KPC, SNL, PTR, STG, RTK, BKV, BLV, KHR 168, ASL, 999 Cash, KTL, SLG, SST, ASY168, and HO).
4. Dim_Account: References Fact_Table, retains GL Code, Description, and Category, removes duplicates on GL Code (producing 78 unique ledger accounts), and adds the Expense Group Conditional Column.

6.2.3 Star Schema Modeling and DAX Programming in Power Pivot

6.2.3.1 Data Model Loading and Relationship Configuration

All four queries were loaded into Excel using Close & Load To... -> Only Create Connection with Add this data to the Data Model checked. Inside the Power Pivot Diagram View, one-to-many (1:∗) relationships were configured linking Dim_Date[Date], Dim_Account[GL Code], and Dim_Branch[Branch] to their respective foreign keys in Fact_Table. Furthermore, in the Power Pivot Data View for Dim_Date, the Month_Name column was sorted by Month_Num using the Sort by Column command so that all monthly charts and slicers display in strict chronological order (January through September).

6.2.3.2 DAX Measure Implementation

Six explicit DAX measures were created in the Fact_Table calculation grid inside Power Pivot to compute MEATOPHUM Microfinance's core financial indicators dynamically across any active filter context:

- Total Income**:** =CALCULATE(SUM(Fact_Table[Amount]), Dim_Account[Category] = "Income")
- Total Expense**:** =CALCULATE(SUM(Fact_Table[Amount]), Dim_Account[Category] = "Expense")
- Net Income**:** = [Total Income] - [Total Expense]
- Profit Margin**:** = DIVIDE([Net Income], [Total Income])
- Income Growth %**:** =DIVIDE([Total Income] - CALCULATE([Total Income], DATEADD(Dim_Date[Date], -1, MONTH)), CALCULATE([Total Income], DATEADD(Dim_Date[Date], -1, MONTH)))
- Expense Growth %**:** =DIVIDE([Total Expense] - CALCULATE([Total Expense], DATEADD(Dim_Date[Date], -1, MONTH)), CALCULATE([Total Expense], DATEADD(Dim_Date[Date], -1, MONTH)))

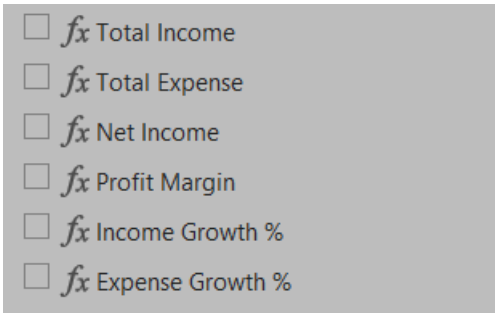


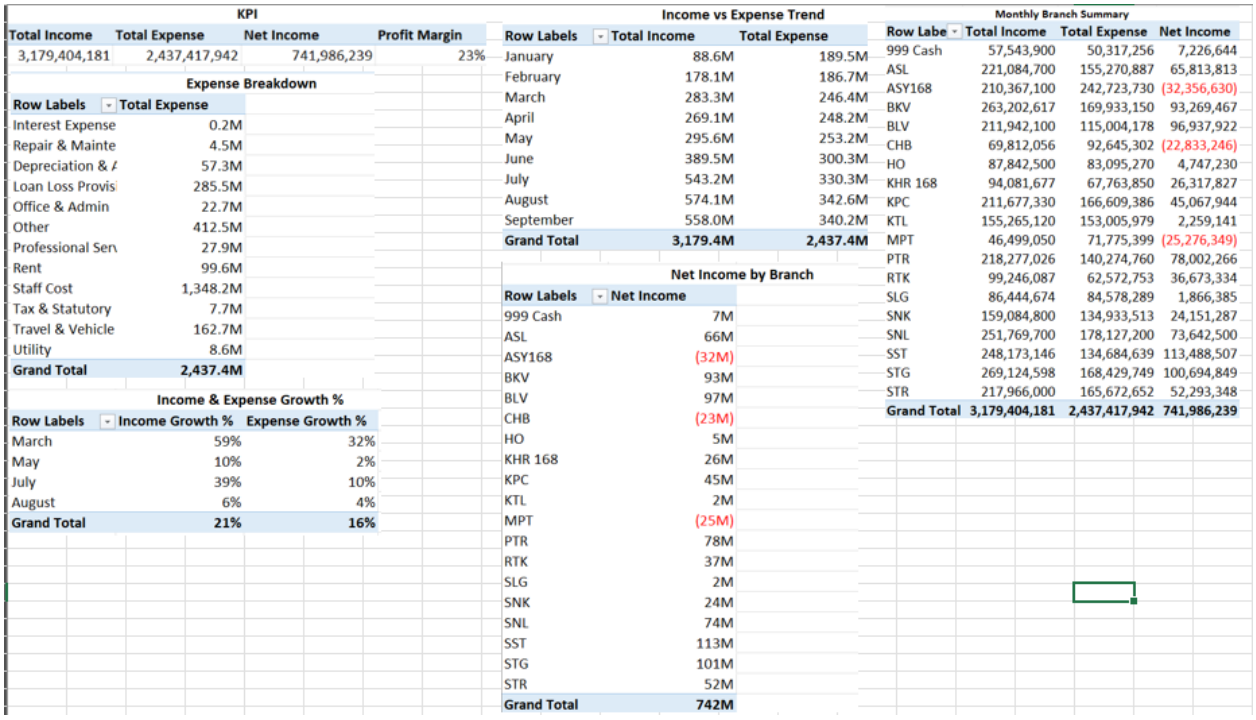
Figure 6.7: DAX Measures Defined in the Power Pivot Calculation Grid

6.3 Workbook Architecture and Dashboard Construction

6.3.1 Analytical Staging Worksheet (Analysis Sheet)

To keep the visual layout stable when filters change, all underlying PivotTables connected to the Power Pivot Data Model were constructed on a dedicated backend worksheet named Analysis. Five specialized PivotTables were configured on this sheet:

1. **Executive KPI PivotTable:** Outputs the grand totals for Total Income, Total Expense, Net Income, and Profit Margin.
2. **Monthly Trend PivotTable:** Aggregates Total Income, Total Expense, and Net Income across Dim_Date[Month_Name].
3. **Expense Group PivotTable:** Aggregates Total Expense across the twelve categories in Dim_Account[Expense Group], sorted from largest to smallest expense driver.
4. **Branch Comparison PivotTable:** Aggregates Total Income, Total Expense, and Net Income across all 19 branches in Dim_Branch[Branch].
5. **Monthly Growth Percentage PivotTable:** Calculates sequential month-over-month percentage changes to track branch growth trends.



KPI

Total Income	Total Expense	Net Income	Profit Margin
3,179,404,181	2,437,417,942	741,986,239	23%

Expense Breakdown

Row Labels	Total Expense
Interest Expense	0.2M
Repair & Mainte	4.5M
Depreciation & A	57.3M
Loan Loss Provis	285.5M
Office & Admin	22.7M
Other	412.5M
Professional Ser	27.9M
Rent	99.6M
Staff Cost	1,348.2M
Tax & Statutory	7.7M
Travel & Vehicle	162.7M
Utility	8.6M
Grand Total	**2,437.4M**

Income & Expense Growth %

Row Labels	Income Growth %	Expense Growth %
March	59%	32%
May	10%	2%
July	39%	10%
August	6%	4%
Grand Total	**21%**	**16%**

Income vs Expense Trend

Row Labels	Total Income	Total Expense
January	88.6M	189.5M
February	178.1M	186.7M
March	283.3M	246.4M
April	269.1M	248.2M
May	295.6M	253.2M
June	389.5M	300.3M
July	543.2M	330.3M
August	574.1M	342.6M
September	558.0M	340.2M
Grand Total	**3,179.4M**	**2,437.4M**

Net Income by Branch

Row Labels	Net Income
999 Cash	7M
ASL	66M
ASY168	(32M)
BKV	93M
BLV	97M
CHB	(23M)
HO	5M
KHR 168	26M
KPC	45M
KTL	2M
MPT	(25M)
PTR	78M
RTK	37M
SLG	2M
SNK	24M
SNL	74M
SST	113M
STG	101M
STR	52M
Grand Total	**742M**

Monthly Branch Summary

Row Labe	Total Income	Total Expense	Net Income
999 Cash	57,543,900	50,317,256	7,226,644
ASL	221,084,700	155,270,887	65,813,813
ASY168	210,367,100	242,723,730	(32,356,630)
BKV	263,202,617	169,933,150	93,269,467
BLV	211,942,100	115,004,178	96,937,922
CHB	69,812,056	92,645,302	(22,833,246)
HO	87,842,500	83,095,270	4,747,230
KHR 168	94,081,677	67,763,850	26,317,827
KPC	211,677,330	166,609,386	45,067,944
KTL	155,265,120	153,005,979	2,259,141
MPT	46,499,050	71,775,399	(25,276,349)
PTR	218,277,026	140,274,760	78,002,266
RTK	99,246,087	62,572,753	36,673,334
SLG	86,444,674	84,578,289	1,866,385
SNK	159,084,800	134,933,513	24,151,287
SNL	251,769,700	178,127,200	73,642,500
SST	248,173,146	134,684,639	113,488,507
STG	269,124,598	168,429,749	100,694,849
STR	217,966,000	165,672,652	52,293,348
Grand Total	**3,179,404,181**	**2,437,417,942**	**741,986,239**

Figure 6.8: Underlying PivotTables Structured on the Analysis Worksheet

6.3.2 Interactive Executive Dashboard (Dashboard Sheet)

6.3.2.1 KPI Cards and PivotChart Layout

The Dashboard worksheet serves as the primary executive interface for the Head of Finance. Along the top banner, four KPI cards display Total Income, Total Expense, Net Income, and Profit Margin by referencing the corresponding summary cells on the Analysis sheet. Below the KPI banner, four interactive PivotCharts visualize institutional performance:

1. **Monthly Financial Trend Chart:** Displays month-by-month trajectories of Total Income, Total Expense, and Net Income from January through September 2026.
2. **Top Expense Categories Chart:** Displays a horizontal breakdown of operating costs across the twelve Expense Group categories.
3. **Branch-Level Performance Comparison Chart:** Compares revenue and operating expenses across all 19 branches.
4. **Month-over-Month Growth Rate Chart:** Visualizes percentage growth fluctuations across reporting periods.

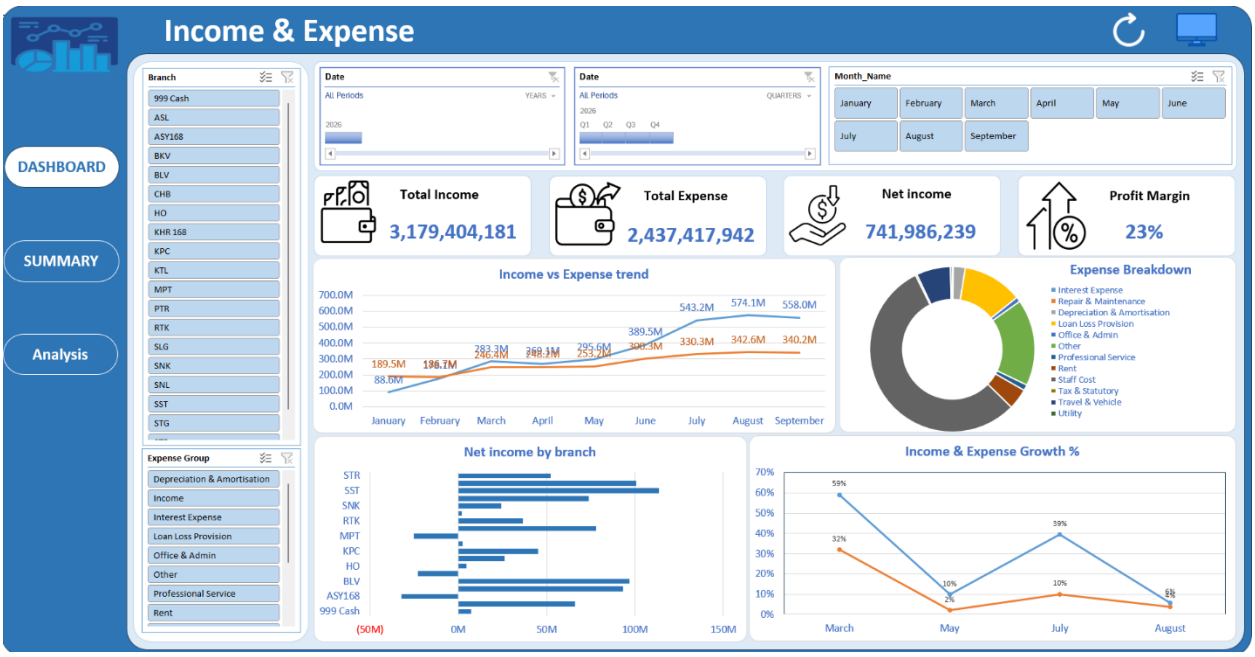


Figure 6.9: Full View of the Completed Interactive Dashboard Worksheet

6.3.2.2 Slicers and Multi-Sheet Report Connections

Interactive Slicers for Branch (Dim_Branch[Branch]) and Month (Dim_Date[Month_Name]) were positioned on the Dashboard sheet and linked to all PivotTables across the workbook via Excel's Report Connections settings. Selecting any combination of branches or months dynamically filters every KPI card, PivotChart, and summary table simultaneously.

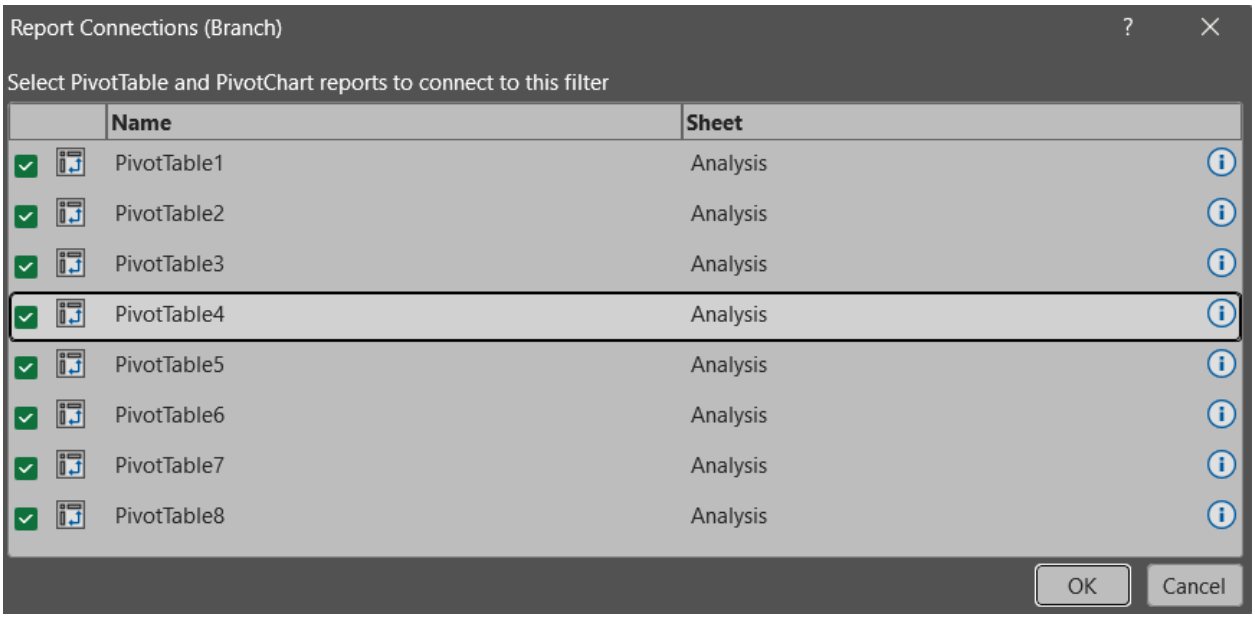


Figure 6.10: Interactive Branch Slicers with Report Connections Dialog

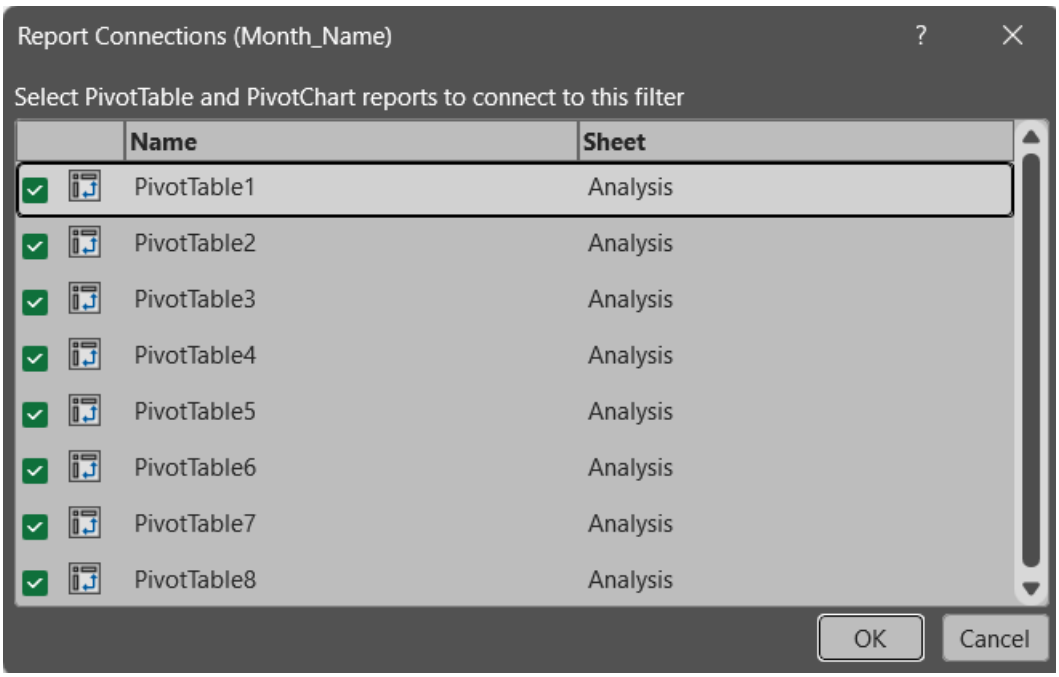


Figure 6.11: Interactive Month Slicers with Report Connections Dialog

6.3.3 Branch-Level Financial Statement (Summary / SUM-Auto Sheet)

Accessed via the SUMMARY navigation button on the Dashboard, the Summary worksheet (named SUM-Auto in the workbook) replicates the branch-by-branch financial summary previously compiled by hand each month in the legacy SUM-IS sheet. It contains an interactive Monthly Branch Summary PivotTable, dedicated Month_Name and Branch slicers, a Branch Performance based on Net Income chart, and a DASHBOARD return button, providing an audit-ready report that updates automatically with the rest of the workbook.

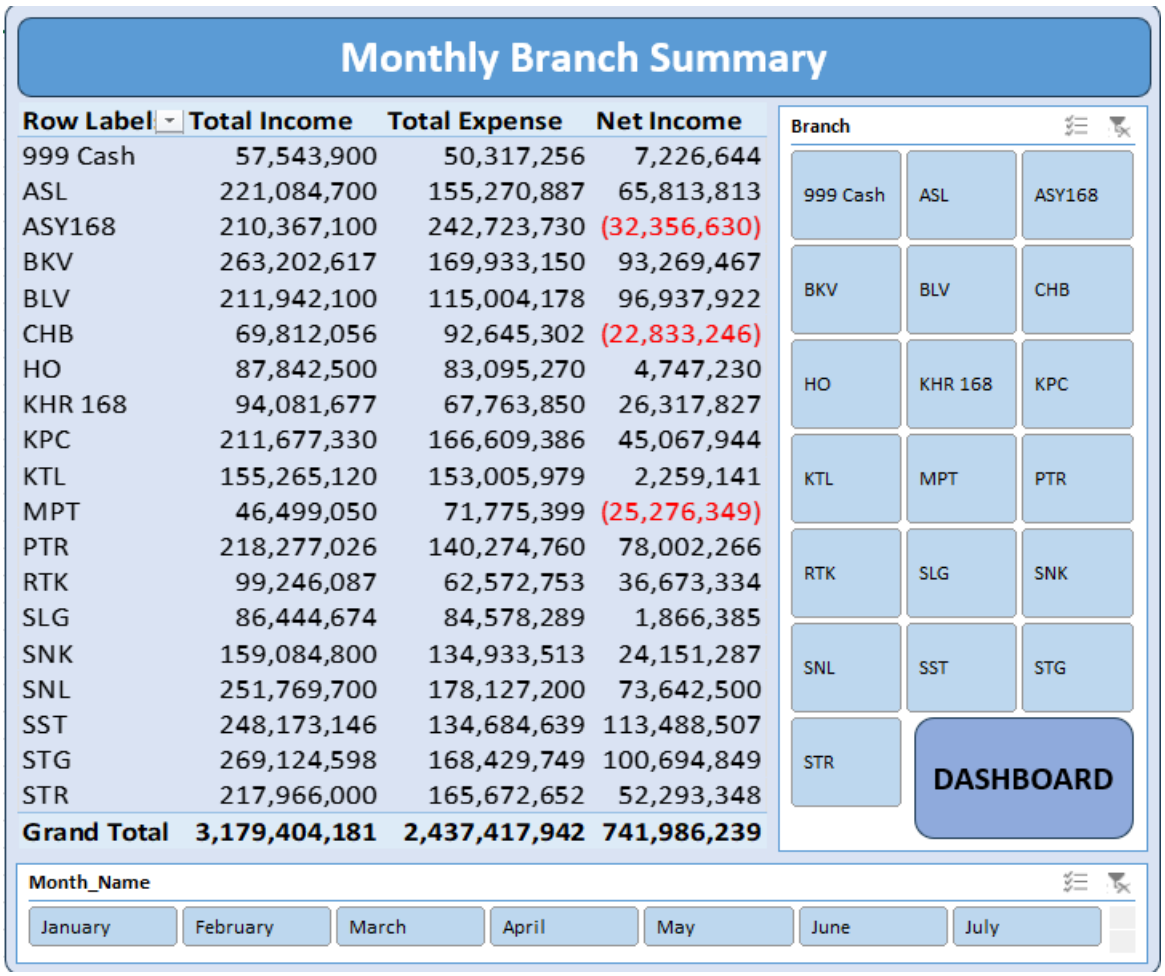


Figure 6.12: Automated Branch-by-Branch Summary on the SUM-Auto Worksheet

6.4 Final Configuration and VBA Automation

6.4.1 VBA One-Click Refresh Implementation (RefreshDashboard)

To enable the Head of Finance to update the entire system with a single click after placing a new monthly file into the Dashboard_Data folder, a VBA macro named RefreshDashboard was written in Module1 inside the macro-enabled workbook (20260531-Income and Expensed-Jan to July 26.xlsm). When triggered, the ThisWorkbook.RefreshAll command instructs Power Query to re-scan the Dashboard_Data directory, append any newly added flat-format monthly files into Fact_Table and the three dimension tables, and recalculate all six DAX measures and eight PivotTables in a single operation.

6.4.2 Image-Based Navigation Buttons and Interface Polish

Clickable rounded-rectangle and image-based buttons were integrated directly into the header area of the Dashboard, Analysis, and Summary (SUM-Auto) sheets. The Refresh button is assigned to the RefreshDashboard macro, while the sheet navigation buttons (DASHBOARD, SUMMARY, and Analysis) use internal workbook hyperlinks (#Dashboard!A1, #'SUM-Auto'!A1, and #Analysis!A1) so the user can switch seamlessly between views without scrolling through the 28 worksheet tabs at the bottom of the workbook. Worksheet gridlines, formula bars, and row/column headings were hidden on the Dashboard to deliver a clean, executive-ready user interface.

CHAPTER 7 RESULTS AND DISCUSSION

This chapter evaluates the operational and analytical outcomes of the automated Income and Expense reporting system implemented for MEATOPHUM Microfinance. The evaluation covers three areas: the data quality and validation results of the Python preprocessing pipeline, the consolidated financial insights generated across the 19 branches from January to September 2026, and a performance comparison between the legacy manual reporting workflow and the automated dashboard solution.

7.1 Data Cleaning and Validation Results

7.1.1 Detection and Exclusion of the Stray Undated Table

When executing Transform_to_raw_data.ipynb on the raw IS-All worksheet in 20260531-Income and Expensed-Jan to July 26.xlsx, the script detected eight candidate 'GL Code' header rows at zero-based row indices [4, 65, 133, 209, 291, 395, 486, 578], but only seven valid 'Date :' banner rows at indices [3, 64, 132, 208, 290, 394, 485] corresponding to January through July 2026.

The custom date-proximity check (0 <= h - d <= 4) successfully validated the first seven monthly blocks and isolated row index 578 in skipped_header_rows. Inspection of row 578 in IS-All confirmed that it was an undated auxiliary note table containing two utility accounts—GL Code 633311101 (Utility Expense-KHR -Elelectric, totaling 840,500 KHR) and GL Code 633311102 (Utility Expense-KHR -Water Supplies, totaling 225,750 KHR)—alongside informal Khmer staff annotations. Excluding this stray block prevented 1,066,250 KHR in duplicate utility expenses from corrupting the July 2026 figures, ensuring 100% reconciliation with the accounting department's verified ledger balances.

7.1.2 Consolidated Dataset Volume and Structure

Following the one-time historical extraction (IS_Master_All.xlsx) and the automated ingestion of August_2026_NewData_FlatFormat.xlsx and September_2026_NewData_FlatFormat.xlsx via Power Query, the Fact_Table in the Power Pivot Data Model consolidated exactly **3,664 non-null transaction rows** spanning **9 months** (January–September 2026), **19 branches**, and **78 unique GL Codes**.

7.2 Financial Performance Analysis (January–September 2026)

7.2.1 Longitudinal Monthly Performance

Across the nine-month reporting window from January to September 2026, MEATOPHUM Microfinance generated a consolidated Total Income of **3,179,404,181 KHR**, a Total Expense of **2,437,417,942 KHR**, and a cumulative Net Income of **741,986,239 KHR**, representing an institutional Profit Margin of **23.34%**. Across the 9 monthly observations, Total Income averaged **353,267,131 KHR** per month (median: **295,617,050 KHR**; standard deviation: **174,675,582 KHR**; minimum: **88,610,600 KHR** in January; maximum: **574,054,200 KHR** in August), while Total Expense averaged **270,824,216 KHR** per month (median: **253,168,731 KHR**; standard deviation: **60,696,722 KHR**; minimum: **186,730,190 KHR** in February; maximum: **342,594,900 KHR** in August).

As shown in Table 7.1, the institution experienced initial operating losses in January (-100,902,227 KHR) and February (-8,639,270 KHR) as branch operations ramped up, before achieving

sustained profitability from March (36,847,486 KHR) through September (217,842,341 KHR), peaking in August with a monthly net profit of 231,459,300 KHR and a 40.32% profit margin.

Month (2026)	Total Income (KHR)	Total Expense (KHR)	Net Income (KHR)	Profit Margin (%)
January	88,610,600	189,512,827	-100,902,227	-113.87%
February	178,090,920	186,730,190	-8,639,270	-4.85%
March	283,257,900	246,410,414	36,847,486	13.01%
April	269,091,300	248,187,845	20,903,455	7.77%
May	295,617,050	253,168,731	42,448,319	14.36%
June	389,474,301	300,343,607	89,130,694	22.88%
July	543,161,600	330,265,459	212,896,141	39.20%
August	574,054,200	342,594,900	231,459,300	40.32%
September	558,046,310	340,203,969	217,842,341	39.04%
Grand Total	**3,179,404,181**	**2,437,417,942**	**741,986,239**	**23.34%**

Table 7.1: Monthly Consolidated Income, Expense, Net Income, and Profit Margin (Jan–Sep 2026)

7.3 System Workflow Evaluation (Manual vs. Automated Process)

Table 7.2 summarizes the operational improvements achieved by replacing the manual SUM-IS consolidation workflow with the automated pipeline.

Evaluation Criterion	Legacy Manual Process (SUM-IS)	Automated Dashboard System (SUM-Auto & Dashboard)
Monthly Data Integration	Manual copy-pasting and retyping across 19 branch columns every month.	Drop the new flat-format monthly .xlsx file into Dashboard_Data and click the Refresh button once.

Evaluation Criterion	Legacy Manual Process (SUM-IS)	Automated Dashboard System (SUM-Auto & Dashboard)
Update Execution Time	Several hours of manual entry and formula checking per monthly cycle.	Under 5 seconds via ThisWorkbook.RefreshAll.
Data Quality & Validation	Vulnerable to transcription typos and silent duplicate blocks (such as the undated utility table at row 578).	Automated Python date-proximity check excludes stray tables; DAX measures guarantee exact ledger reconciliation.
Expense Granularity	Over 60 raw GL descriptions difficult to compare visually.	Automatically grouped into 12 standardized Expense Group categories via Power Query.
Analytical Interactivity	Static spreadsheet table with no interactive filtering or charts.	Interactive KPI cards, 4 PivotCharts, Branch/Month/Expense Group Slicers, Date Timeline, and VBA navigation.

Table 7.2: Operational Comparison Between the Legacy Manual Workflow and the Automated Dashboard Solution

CHAPTER 8 CONCLUSION

This internship project successfully designed, implemented, and validated an automated Income and Expense Dashboard for MEATOPHUM Microfinance using Python and Microsoft Excel. By combining a one-time Python data cleaning script (Transform_to_raw_data.ipynb) with Power Query folder consolidation, a Power Pivot star schema data model, six dynamic DAX measures, and VBA-driven user interface automation, the project replaced an error-prone manual reporting workflow across 19 branches with a self-refreshing business intelligence solution inside Excel.

8.1 Challenges

Several practical challenges were encountered and resolved during the internship:

1. **Inconsistent Raw Export Formatting:** The historical IS-All worksheet stacked multiple monthly blocks vertically with irregular spacing, embedded text banners ("Date : 31th Jan 2026"), and non-standard ordinal suffixes, which prevented direct ingestion into Power Query.
2. **Silent Duplicate Data from a Stray Undated Table:** A leftover utility notes table embedded at row 578 below the July 2026 block contained a valid 'GL Code' header but no date banner, causing naive forward-fill logic to duplicate 1,066,250 KHR of utility expenses under July. Designing the date-proximity validation check (0 <= h - d <= 4) was essential to detect and skip this anomaly automatically.
3. **High Granularity and Typos in Raw Account Descriptions:** The raw chart of accounts contained over sixty expense line items with inconsistent spellings (such as "Communication Expene", "Prefessional Service", and "Elelectric"). Crafting ordered Text.Contains rules in Power Query was required to capture every variant accurately within the twelve standardized Expense Group categories.
4. **Strict Tooling Constraints:** Because the final solution had to run entirely inside Microsoft Excel without Power BI licensing, the workbook had to be carefully engineered using Power Pivot connections, hidden staging PivotTables on the Analysis sheet, and VBA macros so that it behaved like a dedicated BI application for a non-technical user.

8.2 Experience and Lesson Learned

Completing this project within the Finance Department at MEATOPHUM Microfinance provided valuable technical and professional lessons:

1. **Importance of Automated Data Validation:** Real-world organizational spreadsheets rarely conform to textbook schemas. Verifying calculated outputs against source ledgers and building programmatic guardrails—such as the Python date-proximity check—is critical to preventing silent reporting errors.
2. **Power of Dimensional Modeling in Excel:** Structuring the data into a Kimball star schema (Fact_Table connected to Dim_Date, Dim_Branch, and Dim_Account) rather than a single flat sheet allowed six DAX measures to power eight PivotTables and five charts effortlessly across all 19 branches.
3. **User-Centric Dashboard Design:** Technical sophistication is only effective if the end user can operate the tool comfortably. Abstracting the refresh and sheet-switching workflows

behind clickable buttons and ThisWorkbook.RefreshAll ensured immediate adoption by the Head of Finance.

8.3 Further Work

To build upon the foundation established by this project, three future enhancements are recommended:

1. **Direct Database Integration:** When MEATOPHUM Microfinance upgrades its core accounting system, Power Query can be connected directly to the SQL database view rather than intermediate .xlsx exports in the Dashboard_Data folder, eliminating file export steps entirely.
2. **Budget vs. Actual Variance Analysis:** Adding a Fact_Budget table linked to Dim_Branch, Dim_Date, and Dim_Account within the Power Pivot Data Model would enable DAX variance measures to track monthly branch spending against approved annual budgets.
3. **Automated Balance Sheet (**BS-All**) Integration:** Extending the existing Star Schema and Power Query pipeline to incorporate the institution's Balance Sheet (BS-All) and loan portfolio worksheets would provide a unified executive suite covering profitability, liquidity, and credit risk in a single workbook.

REFERENCES

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John Wiley & Sons.

Microsoft. (n.d.-a). *Data Analysis Expressions (DAX) in Power Pivot*. Microsoft Support. https://support.microsoft.com/en-us/office/data-analysis-expressions-dax-in-power-pivot-bab3fbe3-2385-485a-980b-5f64d3b0f730

Microsoft. (n.d.-b). *Getting started with VBA in Office*. Microsoft Learn. https://learn.microsoft.com/en-us/office/vba/library-reference/concepts/getting-started-with-vba-in-office

Microsoft. (n.d.-c). *Power Query documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/power-query/

Microsoft. (n.d.-d). *Understand star schema and the importance for Power BI*. Microsoft Learn. https://learn.microsoft.com/en-us/power-bi/guidance/star-schema

Panko, R. R. (2016). What we don't know about spreadsheet errors today: The facts, why we don't believe them, and what we need to do. *arXiv preprint arXiv:1602.02601*. https://doi.org/10.48550/arXiv.1602.02601

APPENDICES

The complete source code, Python data cleaning notebook (`Transform_to_raw_data.ipynb`), Power Query M scripts, and Microsoft Excel dashboard template developed for this thesis are available upon request. Interested individuals can contact me at novpanhavath070@gmail.com for access, or request collaborator access to my GitHub repository at https://github.com/NPanhavath/automated-income-expense-dashboard